

Applied Biological Data Analysis

Nick Green

2025-08-04

Table of contents

Preface	6
Work in progress warning!	6
Data used in this website	6
License and permissions	7
Course description	7
Course objectives	7
Course requirements	8
Recommended reading	8
Course organization	9
About the author	12
 I Statistics in Modern Biology	 15
1 Statistics in biology	17
1.1 Overview	17
1.2 Statistics in modern biology	17
1.3 How do we use statistics in biology?	19
1.4 P -values and null hypotheses	20
1.4.1 Null and alternative hypotheses	20
1.4.2 Another way to frame null hypotheses	23
1.4.3 History and status of p -values	23
1.4.4 Where p -values come from	24

1.4.5	What p -values mean and do not mean	28
1.4.6	Do you need a p -value?	30
II	Introduction to R	31
2	Introduction to R	33
2.1	Getting started with R	33
2.1.1	Download and install R (and RStudio)	33
2.1.2	Using R	34
2.2	Commands, objects, and output	38
2.2.1	R commands—basics	38
2.2.2	Using R functions	42
2.2.3	The R workspace	44
2.3	R syntax	45
2.3.1	Object size and structure	45
2.3.2	Operators	47
2.4	R packages	50
2.4.1	Your R library	50
2.4.2	Installing R packages	51
2.4.3	Working with packages in R	51
2.5	Frequently asked questions about R	56
2.5.1	What is R	56
2.5.2	Why do we have to write code?	57
2.5.3	Advantages and disadvantages	58
2.5.4	Base R vs. tidyverse	58
2.5.5	Where can I get R help?	60
3	R data types and objects	69
3.1	R data types	69
3.1.1	Character type	69
3.1.2	Numeric type	71
3.1.3	Integer type	72

3.1.4	Logical type	73
3.1.5	Special values	75
3.2	Basic R data structures	77
3.2.1	Vectors	77
3.2.2	Data frames	84
3.2.3	Matrices and arrays	87
3.2.4	Lists	90
3.2.5	S3, S4, and other paradigms	97
4	Data import and export	98
4.1	Working with your file system	98
4.1.1	Your file system	98
4.1.2	Folder addresses	98
4.1.3	The working directory	99
4.2	Formatting data files	101
4.2.1	Naming data files	101
4.2.2	Setting up data files	102
4.3	Data import	106
4.3.1	Importing from text files	106
4.3.2	Importing from saved workspaces	106
4.3.3	Importing special cases	106
4.4	Data export	106
4.4.1	Exporting text files	106
4.4.2	Saving workspaces	106
4.4.3	Exporting graphics	106
5	Data manipulation with R	107
III	Exploratory data analysis with R	108
6	Basic descriptive statistics	110
7	Summarizing biological data	111

8	Visualizing biological data	112
9	Probability distributions for biologists	113
10	Common problems in biological data	114
11	Planning your data analysis	115
IV	Introductory statistics with R	116
12	Introductory statistics with R	118
13	Tests comparing groups	119
14	Tests for continuous patterns	120
15	Tests for contingency	121
V	Generalized linear models (GLM)	122
16	Introduction to generalized linear models (GLM)	124
17	GLM for continuous response data	125
18	GLM for count data	126
19	GLM for binary and proportional responses	127
VI	Extensions of GLMs: nonlinear and mixed models	128
20	Nonlinear models for biology	130
21	Random effects and mixed models	131
VII	Multivariate data analysis	132
22	Introduction to multivariate data and data analysis	134

23 Clustering for biologists	135
24 Ordination	136
25 Multivariate hypothesis testing	137
 VIII Alternatives to p -values: information-theoretic and Bayesian methods	 138
26 Information theoretic model-based inference	140
27 Introduction to Bayesian inference for biologists	141
 IX Bonus material	 142
28 Advanced R graphics	144
 Appendices	 145
A Source and preparation of NEON data	145

Preface

This website accompanies a course I have developed at Kennesaw State University: **BIOL 7410: Applied Biological Data Analysis**. The target audience is biologists who need to analyze data for their research projects. If you find the material here helpful, please let me know! Likewise, if you have comments for improvement, I'd love to hear from you: send all comments to ngreen62@kennesaw.edu!

If you are a returning visitor, please note that the website has been rearranged since 2024. This was done to make the modules smaller and more digestible.

The Fall 2025 course syllabus can be [downloaded here](#).

Work in progress warning!

I am actively updating this site throughout the Fall 2025 semester. This means that many cross-references may be broken, pictures missing, etc. Please be patient with me as I get things back into working order.

Data used in this website

This website presents many examples of data analyses or data manipulation. In past versions of the site (and my course), I have tried to use different and interesting datasets for every module, so that examples would feel more relevant and fresh. However, student feedback has convinced me to pursue the opposite strategy: use a single dataset for most examples, so that people could get more familiar with the dataset over the course of the semester. This would prevent having to learn a completely new dataset and biological system every day in class.

However, it turns out that it's hard to do everything with a single dataset. So, I settled on 3 datasets:

1. The famous `iris` dataset included with R. This will be used primarily in early modules covering data manipulation and data frame management, as well as the review of basic statistics in Module 9.
2. Datasets simulated *ad hoc* to demonstrate simple concepts with no underlying biological reality (i.e., numbers to demonstrate the properties of a probability distribution).
3. Small mammal trapping and measurement data from the [National Ecological Observatory Network \(NEON\)](#) (National Ecological Observatory Network (NEON) 2025). The details of which data I used and how I packaged them for this website can be found in Appendix A.

License and permissions

This work and its content is released under the [Creative Commons Attribution-ShareAlike 4.0](#) license. This means that you are allowed to share the material and adapt it for your purposes, under the following conditions:

1. Give appropriate credit to the [author](#). This includes citing the original sources of any third-party datasets used in some of the tutorials. I've done my best to provide appropriate citations alongside these datasets.
2. If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.

This site is for educational purposes only. This work and its content is released under the [Creative Commons Attribution-ShareAlike 4.0](#) license. Inclusion of third-party data for educational purposes falls under guidelines of fair use as defined in [section 107 of the US Copyright Act of 1976](#).

Course description

This course is a survey of data analysis skills and statistical methods essential for careers in biology. This course takes a holistic approach to the data analysis workflow using the open-source software R, including: data management, exploratory analyses, modeling, and reproducible science practices. Statistical topics covered may include generalized linear models, mixed effects models, nonlinear models, and ordination. Students apply techniques learned in class to real biological datasets as a course project.

Course objectives

1. Engage with current biology literature to explain the role of statistics in the biological sciences and the ways in which analytical results are

communicated.

2. Engage in research using modern statistical methods to investigate biological questions.
3. Manipulate, summarize, display, and analyze data using the open-source environment and language R.
4. Use probability distributions to model and think about biological phenomena.
5. Translate biological hypotheses and ideas into statistical models and vice versa.
6. Conduct exploratory data analyses in support of scientific investigations, particularly to detect and diagnose common problems with biological datasets.
7. Communicate data and analytical results to audiences who may or may not have statistical backgrounds.

Course requirements

- Access to a computer or laptop capable of running [R](#) version 4.0 or above. This site was rendered using R version 4.4.3 and the most recent version of any package mentioned (as of August 8, 2025).
- A 64-bit system is highly recommended for speed and stability.
- A separate program in which to write and edit code.
 - Most people prefer to use an “integrated development environment” (IDE) or dedicated code editor for coding. I recommend some in Module 2.
 - Most people nowadays prefer [RStudio](#). The desktop version is free and has a lot of nice features that make working with R easier.
 - IDEs and code editors are often subject to irrationally strong personal and organizational preferences, so just pick the one that works for you. Or, use the one your supervisor tells you to use.
 - My usual workflow includes two windows: the base R GUI, and [Notepad++](#).
- It would helpful to have a dataset of your own to work with. Each course module uses numerous examples with simulated or published datasets. But, at the end of the day, you’re probably here because you need to apply these methods to your own work.

Recommended reading

This course has no required textbook. If you want a textbook, then I recommend one of the following:

- *Ecological models and data in R* by Ben Bolker (Bolker 2008). This is a general statistics textbook for biologists. The applications and examples are focused on ecology, but the statistical exposition is relevant to any area of biology.
- *Introductory statistics with R* by Peter Dalgaard (Dalgaard 2002). This is a general statistics textbook and introduction to R programming.
- *A beginner's guide to R* by Alain Zuur and others (A. Zuur, Ieno, and Meesters 2009). This book introduces R in a different way than Dalgaard (2002).
- *An introduction to statistical learning: with applications in R* by Gareth James and others (James et al. 2013). This is a more advanced statistics book that also introduces machine learning. The second edition is available for [free online](#).

Parts of the course also lean heavily on some more specialized textbooks:

- *Analysing ecological data* (A. F. Zuur, Ieno, and Smith 2007) and *Mixed effects models and extensions in ecology with R* (A. F. Zuur et al. 2009) by Alain Zuur and others. These are advanced textbooks aimed at ecologists which introduce GLMs, mixed models, and other topics we will explore in this course.
- *Analysis of ecological communities* by Bruce McCune and James Grace (McCune, Grace, and Urban 2002) and *Numerical ecology* by Pierre and Louis Legendre (Legendre and Legendre 2012) are introductions to ordination and other multivariate methods. McCune, Grace, and Urban (2002) is a gentler introduction but depends on [proprietary software](#). Legendre and Legendre (2012) is a little more technical (but still relatively easy to understand) and uses R throughout.
- *An introduction to WinBUGS for ecologists* by Marc Kéry (Kéry 2010). This book introduces Bayesian inference using the free programs WinBUGS or JAGS, which can be accessed from R.

If you want a PDF version of this website, please be patient. I'll put a PDF version here once I can get it to exist.

Course organization

This work is roughly organized into 9 parts. The links below will take you straight to the start of each module.

Part 1 introduces the use of statistics in modern biology, setting the stage for the rest of the course.

Module 1 is an introduction to the use of statistics in biology, and explores some fundamental issues related to how biologists make inferences based on data.

Part 2 is an introduction to R split across several modules. Beginners should start in **Module 2**.

Module 2 is an introduction to the R program and programming language.

Module 3 describes some important R data and object types—how R stores data.

Module 4 covers methods for importing data in R, and exporting results from R.

Module 5 takes a deep dive into manipulating data in R. This includes selecting data, relating data tables, and data frame management.

Part 3 introduces the practices of **exploratory data analysis**, beginning with basic statistical procedures such as descriptive statistics, data summarization, and describing variables using probability distributions. It also includes a module on planning your analysis.

Module 6 describes **descriptive statistics**, or summaries, of single variables.

Module 7 describes how to summarize and aggregate data by groups.

Module 8 introduces basic data visualizations like histograms, probability density plots, boxplots, and scatterplots.

Module 9 introduces probability distributions and how to use them to describe your data.

Module 10 covers common statistical problems in biology, including lack of replication, autocorrelation, collinearity, and missing data.

Module 11 is a guide to selecting statistics for your research. This module contains dichotomous keys to help you decide what statistical methods are most appropriate for the question you are trying to analyze.

Part 4 is a review of introductory statistics—basically everything you probably learned in your undergraduate statistics course, but could use refresher on.

Module 12 reviews the structure of biological datasets, null hypothesis significance testing, and the 4 classes of statistical tests used by biologists.

Module 13 reviews tests for comparing the means or locations of groups of observations: *t*-tests, analysis of variance (ANOVA), and their nonparametric alternatives.

Module 14 reviews procedures for testing continuous patterns: correlation, linear regression, and analysis of covariance (ANCOVA). This module also takes a deep dive into **linear models** and their fundamental importance for understanding more advanced methods.

Module 15 reviews tests for contingency and independence such as Fisher’s exact test, the χ^2 test, and the G -test.

Part 5 introduces the the powerful framework of the generalized linear model (GLM) for modeling for modeling data with a single response variable.

Module 16 introduces generalized linear models (GLM), an extremely general and powerful framework for analyzing biological data.

Module 17 demonstrates GLMs for continuous data: Gaussian, log-normal, and Gamma GLMs.

Module 18 covers GLMs for count data: Poisson and negative binomial GLMs.

Module 19 covers GLMs for binomial data: logistic regression and binomial GLM.

Part 6 builds upon the GLM framework to introduce nonlinear models and mixed models, which can accomodate many situations where GLMs are inadequate.

Module 20 explores nonlinear models for relationships that don’t follow a straight line. These models include biological growth curves and dose response curves.

Module 21 introduces mixed models, which extend GLM and other models to include random effects. Mixed models allow biologists to account for variability from unknown sources.

Part 7 covers multivariate statistics, methods which can consider many variables at a time.

Module 22 introduces **multivariate** statistics, which explore patterns in 2 or more variables at once. This module covers the structure of multivariate datasets and distance measures.

Module 23 explores **cluster analysis**, a class of techniques for identifying groups within multivariate data.

Module 24 explores **ordination**, a class of techniques for visualizing patterns in multivariate data. Ordination also includes **dimension reduction**, or the consolidation of information from many dimensions into few.

Module 25 describes some multivariate hypothesis tests, many of which are extensions of univariate techniques.

Part 8 introduces two statistical frameworks that can be an alternative to p -values and null hypothesis significance testing.

Module 26 introduces information-theoretic model selection, an alternative to null hypothesis significance testing (NHST) that evaluates entire models rather than individual parameters and their p -values.

[Module 27](#) introduces Bayesian inference using the program JAGS, another alternative to NHST.

Part 9 is “bonus material”; i.e., additional modules that didn’t quite fit in anywhere else. New modules will be added here until the next major update.

[Module 28](#) takes a deep dive into building quality graphics using base R.

Finally, Appendix A describes the NEON dataset that I used to build most of this course, and the steps taken to process the raw data download to the files provided here.

About the author



I’m a [quantitative ecologist at Kennesaw State University](#) who studies human impacts on animal populations. Since earning my PhD at Baylor University in 2012, I’ve been fortunate to have worked in a variety of government and industry

roles. Those experiences have shaped my perspective on biology and education. In my courses and advising I do my best to impress upon students the variety of biological experiences and perspectives that different domains bring to the table...and the value that they all bring. Most of my [current research](#) focuses on understanding human impacts on animal communities and how animal populations adapt to rapidly changing environments.



Why am I making this guide? I have been using R since about 2008, when I was a graduate student trying to make a multi-panel plot for a manuscript. I had seen R mentioned in publications and thought that using R would be more effective than kludging something together with Excel and Powerpoint. Needless to say, jumping straight to making plots with the `lattice` package was not the easiest way to learn R. The final result was a little clunky (Kirchner et al. 2011) but I am still really proud of it. Eventually I took some courses and did a lot of self-teaching, and am finally (March 2025) pretty decent at R. I have used R in my work pretty much continuously since 2008 for everything from Bayesian data analysis (e.g., Green et al. 2020) to simulation modeling (e.g., Green, Wildhaber, and Albers 2019).

This guide is meant to be a primer for self-starting with R. In many sections the text is a bit lengthy, but that is because I wanted to provide explanations that might save someone some of the trouble I had getting started. R documentation is notoriously terse and compact—this is great for professionals who need a reminder, but not so much for newcomers who are still trying to learn. This guide also evolved out of course notes, and will continue to evolve based on feedback

from students. If you have any feedback or suggestions for improvement, please [send me a message!](#)

Part I

Statistics in Modern
Biology

Do we have to?

Chapter 1

Statistics in biology

1.1 Overview

Science works by testing ideas to see if they line up with observations. In other words, checking the predictions of hypotheses against *data*. This process is called **data analysis**, because it involves careful examination of numerical patterns in observational or experimental data. The branch of mathematics concerned with analyzing data is called **statistics**. Statistics is vital to modern science in general, including biology.

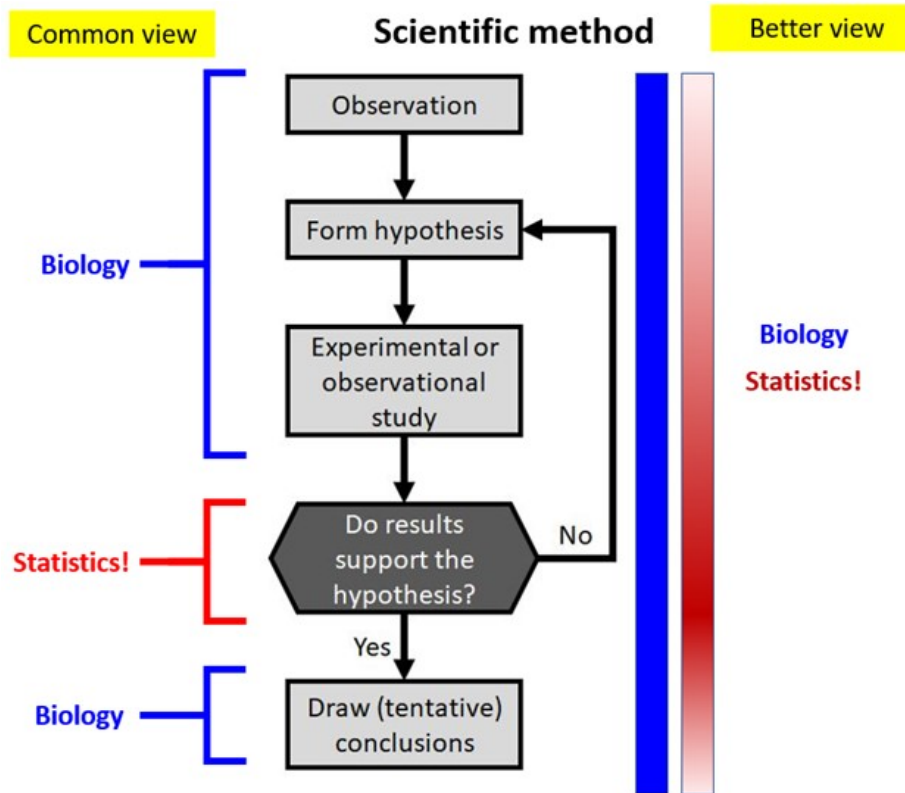
This website and the course it supports is designed to introduce the ways that statistics are used in biology. This includes strategies for data analysis and an introduction to the open source program and language [R](#). Specific tasks in the data analysis workflow (e.g., data manipulation) and specific analysis methods (e.g., generalized linear models) are covered in their own modules. This first module is a more general exploration of the role of statistics in modern biology, common statistical frameworks, and common misuses of statistics.

1.2 Statistics in modern biology

Biology, like all sciences, depends on the **scientific method**. This means testing proposed explanations, or **hypotheses**, against the data to see if the data support the hypothesis or not. That critical step, of deciding whether observations support or refute a hypothesis, is the domain of **statistics**: the branch of mathematics concerned with finding and interpreting patterns in data.

Many biologists traditionally think of “biology” and “data analysis” as two completely separate enterprises, with statistics being like a special wrench that a mechanic pulls out of the toolbox so infrequently that its use is unfamiliar. I

think this is a mistake. As the figure below shows, every part of the scientific method can benefit from some knowledge of statistics. This includes experimental design, data collection, and interpreting the findings.



So why do we, as biologists, need statistics? Some common answers might include:

1. "To tell us whether the patterns we see in our data are real"
2. "To tell whether the patterns in our data are due to chance or not."
3. "To prove our hypotheses."

All 3 of these are completely wrong. **1** is wrong because confirming reality requires observation and experiment, not calculating. **2** is a common misconception. *Any* pattern in data could be caused by chance. Statistics can help you estimate that probability, but cannot tell you for sure. Finally, **3** is wrong because that's not how science works. Hypotheses can be supported or not supported. No amount of statistical analysis will ever completely rule out chance.

So the answer to the question, “Why do we as biologists need statistics?” is that we need a systematic way to determine whether data support hypotheses are not. Note that I said “systematic”, not “absolutely correct”. As we will see shortly, statistical tests are only a guide to decision-making. We should not fool ourselves into thinking that our analyses are completely objective. Many aspects of the analysis process are in fact *subjective* and thus any analysis has to be taken in its proper context, with caveats and biological reality always in mind.

When we “do statistics”, we are arguably not even answering biological questions. Every statistical test answers a particular form of question:

- “Do groups A and B have the same mean?”
- “As variable X changes, does variable Y tend to change?”
- “If variable X increases by this much, how much does variable Y change?”
- “If event G happens, how likely is it that event H also happens?”

Notice that these are “statistical” statements, not necessarily biological statements. Translating biological ideas and questions into statistical statements and questions is one of the hardest parts of “doing statistics”.

1.3 How do we use statistics in biology?

Although there are hundreds, if not thousands, of statistical tests out there, most biologists only ever need a handful. It turns out that statistical hypotheses in biology tend to fall into just a few categories:

1. Testing for a **difference in mean or location** between groups.
2. Testing for a **continuous relationship** between a response variable (or multiple variables) and one or more predictor variables. Also referred to as **predicting** or **modeling** a continuous variable.
3. Testing for (lack of) **independence between categories** of observations.
4. Trying to **classify observations** based on predictor variables.

These categories are not mutually exclusive, and many biological questions can be phrased in more than one way. Understanding what kind of pattern you looking for—that is, which of the 4 categories above your question falls into—can get you a long way toward understanding how to analyze your data.

1.4 *P*-values and null hypotheses

1.4.1 Null and alternative hypotheses

When scientists design experiments, they are testing one or more hypotheses, or proposed explanations for a phenomenon. Contrary to popular belief, the statistical analysis of experimental results usually does not test a researcher’s hypotheses directly. Instead, statistical analysis of experimental results focuses on comparing experimental results to the results that *might have occurred* if there was no effect of the factor under investigation.

In other words, traditional statistics sets up the following:

- **Null hypothesis or $H[0]$:** default assumption that some effect (or quantity, or relationship, etc.) is zero and does not affect the experimental results.
- **Alternative hypothesis or $H[a]$:** assumption that the null hypothesis is false. Usually this is the researcher’s scientific hypothesis, or proposed explanation for a phenomenon.

In (slightly) more concrete terms, if researchers want to test the hypothesis that some factor X has an effect on measurement Y , then they set up an experiment comparing observations of Y across different values of X . They would then have the following set up:

- **Null hypothesis:** Factor X does not affect Y .
- **Alternative hypothesis:** Factor X does affect Y

Notice that in NHST the alternative hypothesis is the **logical negation** of the null hypothesis, ***not*** a statement about another variable (e.g., “Factor Z affects Y ”). This is an easy mistake to make and one of the weaknesses of NHST: the word “hypothesis” in the phrase “null hypothesis” is not being used in the sense that scientists usually use it. This unfortunate choice of vocabulary is responsible for generations of confusion among researchers¹.

In order to test an “alternative hypothesis”, what need is to *estimate* how likely the data would be if the null hypothesis were true. If a pattern seen in the data is highly unlikely, this is taken as evidence that the null hypothesis should be rejected in favor of the alternative hypothesis. This is not the same as “accepting” or “proving” the alternative hypothesis.

The null hypothesis is usually translated to a prediction like one of the following:

¹Not to mention among generations of students taking statistics courses, myself included.

- The mean of Y is the same for all levels of X .
- The mean of Y does not differ between observations exposed to X and observations not exposed to X .
- Outcome Y is equally likely to occur when X occurs as it is when X does not occur.
- Rank order of Y values does not differ between levels of X .

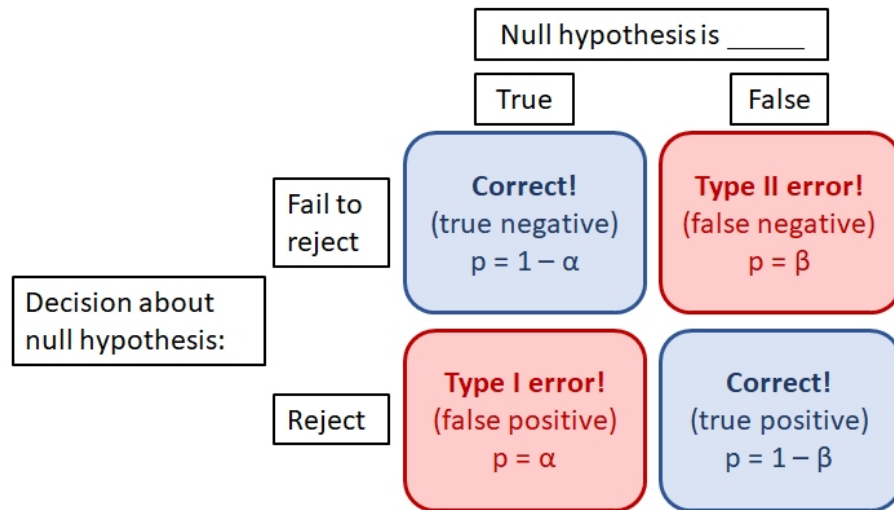
All of these predictions have something in common: Y is **independent** of X . That's what the null hypothesis means. To test the predictions of the null hypothesis, we need a way to calculate what the data *might have looked like if the null hypothesis was correct*. Once we calculate that, we can compare the data to those calculations to see how “unlikely” the actual data were. Data that are sufficiently unlikely under the assumption that the null hypothesis is correct—a “significant difference”—are a kind of evidence that the null hypothesis is false. If this sounds convoluted, that's because it is.

This kind of analysis is referred to as **Null Hypothesis Significance Testing (NHST)** and is one of the most important paradigms in modern science. Whether or not this is a good thing is a subject of fierce debate among scientists, statisticians, and philosophers of science. For better or worse, NHST is all but the *de facto* standard of inference in scientific publishing, so regardless of its merits in your particular case you will need to be able to use it. The purpose of this course module is to encourage you to use it correctly.

The p -value allows us to make binary distinctions, in a systematic way, between situations where a null hypothesis is true and no pattern is present, and situations where the null hypothesis is highly unlikely to be true and thus a pattern might be present. Any binary decision process has four possible outcomes:

1. **True positive:** declare a pattern is present when it is present.
2. **True negative:** declare a pattern is absent, when it is absent.
3. **False positive:** declare a pattern is present, when it is actually absent.
Also called a **Type I error**.
4. **False negative:** declare a pattern is absent, when it is actually present.
Also called a **Type II error**.

These outcomes can be summarized as shown below. Statisticians are particularly concerned with the probabilities of false positives (α) and false negatives (β). The false positive probability α is the standard against which experimental p -values are compared. This is largely because we humans are inordinately prone to finding patterns even when no patterns are present—i.e., committing type I errors.



Why have we evolved this way? Finding patterns in noisy data has helped us discover virtually everything we know. So, pattern recognition must be a powerful cognitive tool. But there is another interpretation, that in our ancestral lifestyle life false positives were far less costly than false negatives. Imagine you are an Australopithecine or an early *Homo erectus* (or *H. sapiens*, for that matter), wandering through the savanna of what will someday be modern Ethiopia, looking for food. You hear a rustle in the tall grass a few meters ahead. Is that rustle just the wind? Or is it a dangerous animal that will kill and possibly eat you? Here are your choices and their consequences:

Your choice:	True source of the noise	
	Just the wind	Dangerous animal
Ignore the noise and keep foraging.	True negative No worries! You get to keep foraging a while longer.	False negative! Type II error! You are killed and eaten by a lion. And if you are very, very lucky...they'll do it in that order.
Run away from the possible lion!	False positive! Type I error! You spend some calories running away, and lose some foraging time, but at least you're alive.	True positive You run for your life and live to forage another day. Everyone at the camp totally believes your story about the lion.

Notice that of your choices, only the type II error has immediate evolutionary consequences. In terms of fitness, the difference between a type II error and a true positive (death vs. survival) is much greater than the difference between a true negative and a false positive (a bit more food vs. a mild inconvenience).

What does this have to do with statistics in biology? Our human minds have evolved to find patterns in noisy data, and we are so good at it that we frequently

“find” patterns that aren’t really there. So, we use the scientific method and statistical analysis as tools to help mitigate our bias toward pattern-finding.

1.4.2 Another way to frame null hypotheses

When I teach statistics to biology majors, I try to avoid the terms “null hypothesis” and “alternative hypothesis” because most of my students find them confusing. Instead, I ask students to ask two questions about a hypothesis:

1. What should we observe if the hypothesis *is not* true?
2. What should we observe if the hypothesis *is* true?

In order to be *scientific* hypothesis, both of those questions have to be answerable. Otherwise, we have no way of distinguishing between a situation where a hypothesis is supported, or a situation where it is not, because experimental results could *always* be due to random chance (even if the probability of that is *very* small). All we can really do is estimate how likely it is that random chance drove the outcome².

1.4.3 History and status of p -values

The p -value is a statistical statement about the likelihood of observing a pattern in data when the null hypothesis is true. They were first developed in the 1700s, and popularized by the statistician R.A. Fisher in the 1920s. In Fisher’s formulation, the p -value was only meant as a “rule-of-thumb” or **heuristic** for determining whether the null hypothesis could be safely rejected. His original publications did not imply that rejecting the null hypothesis should automatically imply acceptance of the alternative hypothesis, or offer any justification for particular cut-offs or critical values of p . Those ideas came later.

Fisher suggested a critical p -value of 0.05 for convenience, but did not prescribe its use. In fact, he also suggested using other cut-offs, and even using the p -value itself as a measure of support for a hypothesis. The adoption of 0.05 as a threshold for significance proved popular, and has been taken as a kind of gospel for generations of researchers ever since.

In my opinion, the ubiquity of p -values on scientific research illustrates [Goodhart’s Law](#): “When a measure becomes a target, it ceases to be a good measure”. Because the cut-off value of $p < 0.05$ is so widely used and accepted as a measure of the “truth” of a hypothesis, many researchers will take steps to ensure that

²You may have noticed that questions 1 and 2 are really the null hypothesis and alternative hypothesis in disguise.

their experiments produce p -values < 0.05 . This can lead to the practices of p -hacking or data dredging. Or worse, setting up experiments to test meaningless null hypotheses or analyzing data selectively.

The attention and effort spent increasing the likelihood of a significant outcome is attention and effort that is not spent on formulating hypotheses, developing theory, or conducting experiments. Thus, intensive focus on p -values can be a real distraction from more important things.

These criticisms of p -values are neither new nor little known. Criticisms of the uncritical devotion to p -values can be found going back decades in the literature. In ecology, for example, debates about the place of NHST flare up occasionally, as new advances in statistical methods have offered opportunities for new inferential paradigms.

Recently, the American Statistical Association (ASA) published a number of papers criticizing the current use of p -values and offering alternatives. Wasserstein and Lazar (2016) provide a good starting point to the series and an overview of the major themes. Some of the proposed alternative or complementary methods have started to make their way into various scientific disciplines. Others have not. By and large, NHST remains the standard method of testing hypotheses with data and will likely remain so for a long time.

1.4.4 Where p -values come from

So where do p -values come from, anyway? Remember, p -values are an expression of how likely the observed data would be if the null hypothesis was true. Then calculation of a p -value involves assumptions about the size of the effect (i.e., difference between the results if the null is true vs. not true), the variability of the outcome (random or otherwise), and the distribution of different summary statistics of the results assuming that the null hypothesis is true. Let's consider the example of a t -test. A t -test is used to compare means of two groups. We'll talk more about them in Chapter 12. We will use R to simulate some data and explore a basic t -test.

```
set.seed(123)
n <- 10
mu1 <- 6
mu2 <- 8
sigma <- 3
x1 <- rnorm(n, mu1, sigma)
x2 <- rnorm(n, mu2, sigma)
t.test(x1, x2)
```

Welch Two Sample t-test

```

data:  x1 and x2
t = -1.796, df = 17.872, p-value = 0.08941
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
-5.2131465  0.4091686
sample estimates:
mean of x mean of y
6.223877  8.625866

```

The results show us that the means of groups **x1** and **x2** are not significantly different from each other ($t = -1.796$, 17.87 d.f., $p = 0.0894$). Because $p \geq 0.05$, we cannot reject the null hypothesis of no difference; i.e., the hypothesis that the means of **x1** and **x2** are the same. This is **not** the same as concluding that the means are the same. All we can say is that we cannot say the means are different.

But why? Where do the numbers in the output come from?

The t value is what's called a **test statistic**. This is a summary of the quantity of interest—difference in means—that depends on the quantity of interest *and* additional terms that describe how that summary might vary randomly. In the case of the t -test, the test statistic t is calculated as:

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$$

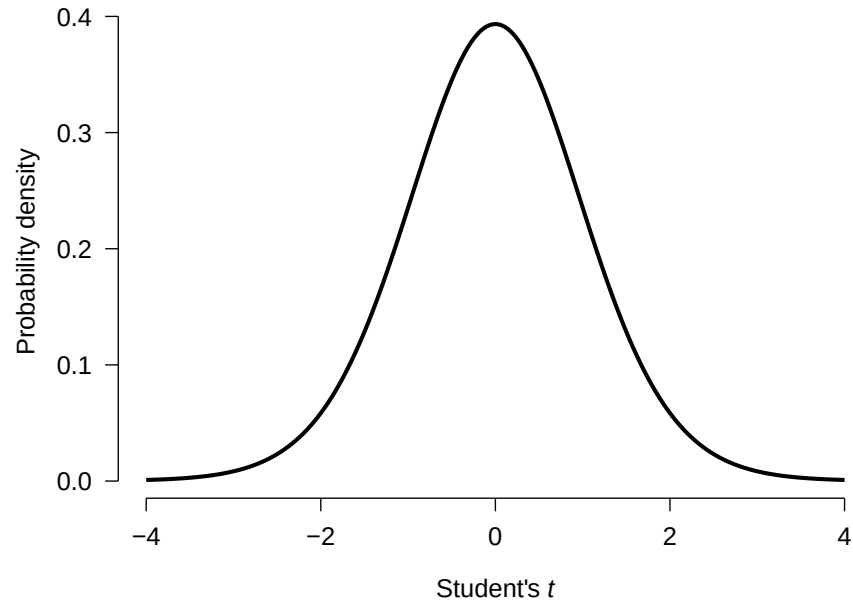
In this equation:

- $\bar{x}_1$ and $\bar{x}_2$ are the means of two sets of values x_1 and x_2 . A bar over a variable usually denotes a sample mean.
- s_1^2 and s_2^2 are the sample variances of x_1 and x_2 . The **variance** is a measure of how spread out values are away from the mean.
- n_1 and n_2 are the number of observations in x_1 and x_2 .

The numerator is the quantity we are interested in: the difference between the means in group 1 and group 2. The denominator scales the difference according to the sample size and the amount of variation in both groups. Thus, t accounts for the quantity of interest (the pattern), the variability in the pattern, and the number of observations of the pattern.

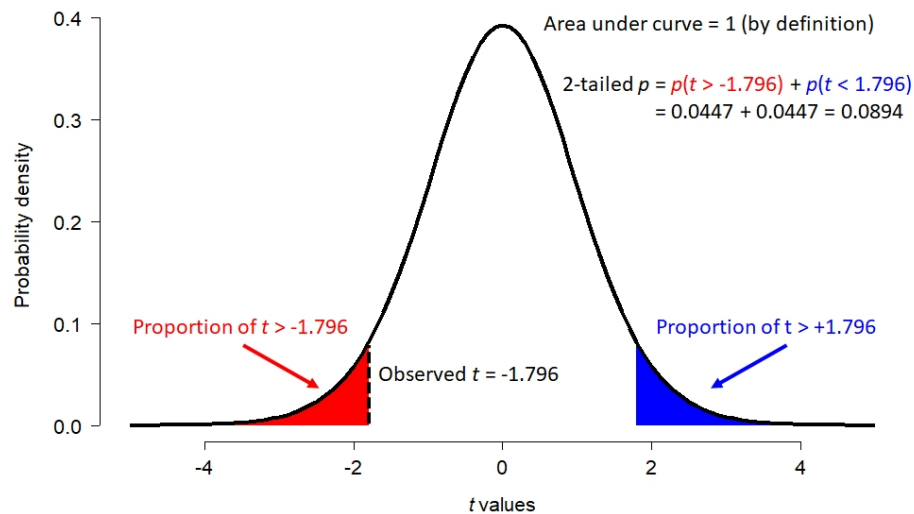
In the example above, the t statistic was -1.796. This was the difference in means (-2.402) scaled by the amount of random variation (s_1^2 and s_2^2) relative to the number of observations (n_1 and n_2). The “significance” of the test comes from the fact that t -statistics themselves follow a pattern called the **Student's t distribution**. The Student's t distribution describes how likely different values

of t are given a certain sample size. This sample size used is not the total number of observations, but instead a scaled version of sample size called **degrees of freedom (d.f.)**. For the example above with 17.872 d.f., the t distribution looks like this:



If there was no difference between the means of x_1 and x_2 , then t statistics of randomly generated datasets should cluster around 0 with more extreme values being much less likely. Test statistics ≥ 4 or ≤ -4 almost never happen.

What about our t statistic, -1.796? The figure below shows how the total possible space of t statistics (given 17.872 d.f.) is partitioned. The total area under this curve, called a “probability density plot”, is equals 1. If you think about it, t must take on a value...so the sum of probabilities for all t is 1. The red area shows the probability of a t occurring randomly that is < -1.796 . That is, more extreme than the observed t . But “more extreme” can work the other way too: t could randomly be > 1.796 as well. This probability of this possibility is shown by the blue area. The total of the red and blue areas is the total probability of a t value more extreme than the observed t .



But as we shall see, increasing the sample size can make the same difference appear significant:

```
set.seed(123)
n <- 1000
mu1 <- 6
mu2 <- 8
sigma <- 3
x1 <- rnorm(n, mu1, sigma)
x2 <- rnorm(n, mu2, sigma)
t.test(x1, x2)
```

Welch Two Sample t-test

```
data: x1 and x2
t = -15.485, df = 1997.4, p-value < 2.2e-16
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
-2.342319 -1.815705
sample estimates:
mean of x mean of y
6.048384 8.127396
```

We can also find a significant p -value by reducing the amount of variation (σ , or σ). In this example, variance within each group is expressed as the

residual standard deviation (SD). **Residual** variation is variation not explained by the model (i.e., not related to the difference in means between groups). The SD is the square root of the variance, and is how R expresses the variation in a normal distribution (`rnorm()`).

```
set.seed(123)
n <- 10
mu1 <- 6
mu2 <- 8
sigma <- 0.001
x1 <- rnorm(n, mu1, sigma)
x2 <- rnorm(n, mu2, sigma)
t.test(x1, x2)
```

Welch Two Sample t-test

```
data: x1 and x2
t = -4486.7, df = 17.872, p-value < 2.2e-16
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -2.001071 -1.999197
sample estimates:
mean of x mean of y
 6.000075  8.000209
```

So, what the P -value in our t -test tells us depends on not just the actual difference, but how variable that difference is and how many times we measure it.

The table below summarizes the results of the three simulations we just ran:

True difference ($\bar{x}_1 - \bar{x}_2$)	Residual SD	Sample size	p -value	Significant?
-2	3	20	0.0894	No
-2	3	2000	<0.0001	Yes
-2	0.001	20	<0.0001	Yes

1.4.5 What p -values mean and do not mean

In the simulations above, we know that the true difference was -2 because that is how we generated the numbers. But the conclusion of the experiments depended on experimental parameters other than the one of interest! For each of the three scenarios above, consider these questions:

- Should the researchers reject the null hypothesis, and conclude that there is a difference between $\mathbf{x1}$ and $\mathbf{x2}$?
- Should the researchers accept the null hypothesis, and conclude that there is no difference between $\mathbf{x1}$ and $\mathbf{x2}$?
- Should the researchers accept the alternative hypothesis, and conclude that there is a difference between $\mathbf{x1}$ and $\mathbf{x2}$?

The researchers should reject the null hypothesis only in scenarios 2 and 3. In scenario 1, they should not “accept” the null hypothesis. The correct action is **fail to reject** the null. Accepting the null hypothesis would imply some certainty that the difference was 0. But that is not what the t -test actually says. What the t -test really says is that a difference (measured as t) between -1.796 and 1.796 is likely to occur about 91% (i.e., $1-P$) of the time even if the true difference is 0. In other words, if we repeated the experiment, we would get a $|t| > 1.796$ over 9 times out of 10!

One of the most common misconceptions about p -values is that they represent the probability of the null hypothesis being true, or of the alternative hypothesis being false. **Neither of these interpretations is correct.** Similarly, many people think that 1 minus the p -value represents the probability that the alternative hypothesis is true. This is also incorrect.

Remember:

- The p -value is only a statistical summary, and says nothing about whether or not a hypothesis is true.
- The p -value is only a statement about how unlikely the result would be if there was no real effect.
- A p -value is a purely statistical calculation and says nothing about biological importance or the truth of your hypothesis.
- Interpreting p -values and other statistical outputs is the job of the scientist, not the computer.

If it feels like I’m belaboring this point, it’s because I am. The p -value and NHST in general are merely a set of heuristics to help researchers make binary distinctions like:

- Reject or fail to reject the null hypothesis.
- True difference is 0 or not 0.
- Difference is ≥ 3 or not <3 .
- And so on.

These distinctions are binary because each condition is the logical negation of the other. But, *rejecting* one side of each dichotomy is *not the same as accepting* the other. By analogy, juries in criminal trials in the USA make one of two

determinations: guilty or not guilty. A verdict of not guilty includes innocent as a possibility, but does not necessarily mean that the jury thinks the defendant is innocent. It simply means that the jury does not feel that the defendant has been *proven* guilty.

Likewise, the p -value might help distinguish “difference in means is different from 0 ($p < 0.05$)” vs. “difference in means is not different from 0 ($p \geq 0.05$)”. The latter choice includes a difference in means of 0 as a possibility, but does not actually demonstrate that the value is 0.

1.4.6 Do you need a p -value?

Now that we have a better handle on what a p -value really is, you might be tempted to ask, “Do I need a p -value?” Yes, you probably do. For better or for worse, p -values and NHST are firmly entrenched in most areas of science, including biology. Despite its shortcomings, the NHST paradigm does offer a powerful way to test and reject proposed explanations. This utility means that NHST and p -values are not going away any time soon. If you want an alternative to p -values and NHST, refer to [Chapter 26](#) or [Chapter 27](#).

Part II

Introduction to R

Why. Doesn't. It. Work?

Chapter 2

Introduction to R

This module is an introduction to the R program and language. We will begin with downloading and installing R, then explore the basics of running R commands and storing data, and finally answer some frequently asked questions (FAQ). Some of the topics in this module are covered in greater detail in Chapter 3, Chapter 4, and Chapter 5. The current module is meant to get you up and running before moving on to more advanced material.

By the end of this module you should be able to:

- Explain what R is and what it is used for
- Explain the object-oriented programming paradigm used by R
- Download and install R and RStudio
- Write and execute commands in the R console
- Describe basic R data types and structures
- Manage R code as scripts (.r files)
- Manage and use R packages
- Identify sources of help for R programming

2.1 Getting started with R

2.1.1 Download and install R (and RStudio)

If you are on a university campus, many on-campus computers may already have R installed on them. This is the bare minimum you need to work with R. Many people also prefer to use another program for writing R code. The most popular is probably [RStudio](#), which is a more user-friendly interface for R. If you are on a PC or Mac, you will need to download and install both R and RStudio. If you are using a Linux machine, you may already have R because

it is included in some distros. You can still install RStudio to make your life easier.

First, install the latest version of R from the [R-project website](#) (accessed 2021-12-20). Select a mirror (download server) from the list. Once you select a mirror, you will be taken to a page where you will select the version of R that corresponds to your operating system. Click on the appropriate link for your operating system and follow the instructions from there to install R. Like many programs you download from the internet, R will come with an installer that will largely take care of everything automatically. Just let the installer run and accept the default options.

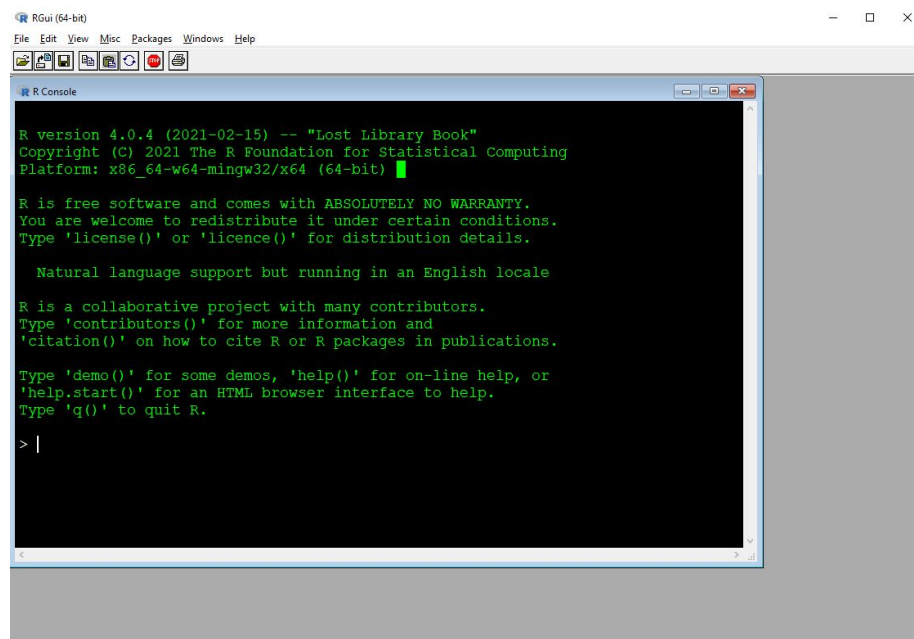
Next, install any code editing or IDE you want.

- RStudio can be installed from [here](#) (accessed 2025-08-06). Select the version that corresponds to your operating system. As you did when you installed R, just let the installer run and accept the default options.
- My personal favorite is Notepad++, which isn't an IDE but instead a very feature-rich code editor.
- Other popular choices include [Visual Studio Code](#) and [emacs](#).

2.1.2 Using R

2.1.2.1 Base R GUI

R has a very basic front end, or **graphical user interface (GUI)**. Most users find it easier to write their R code in another program. Some of these programs, such as RStudio, are **integrated development environments (IDE)**. Others are text editors with programming capabilities. We'll explore the default R GUI first and then meet some editor options. The basic R GUI is shown below. Your screen may look different than mine because I set some graphics options.



The main window with R is the **R console**. This is the command line interface to R. Some important features of the console are:

1. Command prompt: The `>` symbol signifies the beginning of a command. You can enter commands here.
2. Continue prompt: `+` signifies the continuation of a previous command. If you have a `+` at the beginning of your line, you can get back to a new line using `ESCAPE`. Note that this will delete the currently incomplete line or command.
3. The `ENTER` key executes the current command. This could mean the current line, or multiple lines going back to the most recent `>`.
4. `[1]` at the beginning of results: because your result is an object (with length ≥ 1). The very first value (“element”) in the result is element 1.
5. Up arrow key: cycles through previously entered commands. You can see many previous commands at once by using the command `history()`.

Other important features of the R GUI are found in the menu bar:

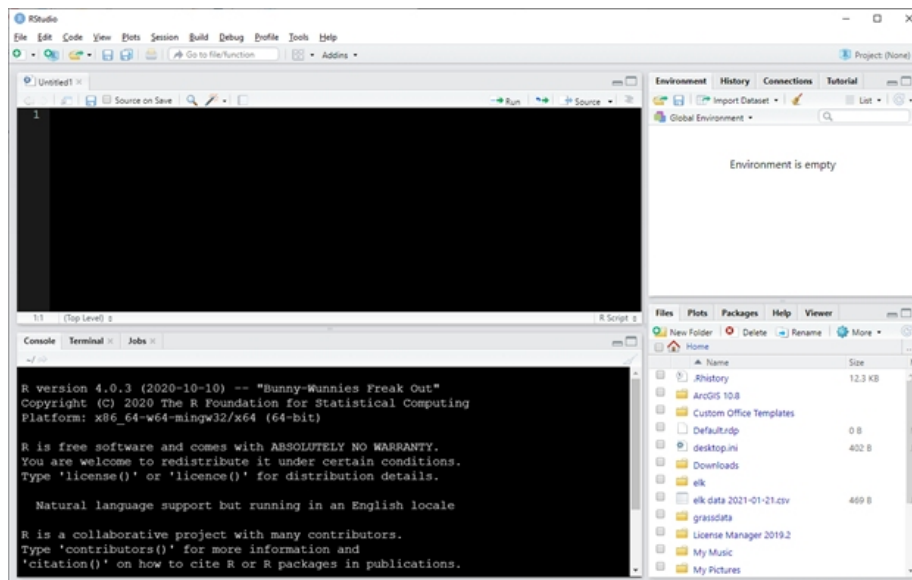
- **File**
 - **New script**: opens a new script window within the GUI.

- Ability to open saved scripts or load previous workspaces
- **Edit**
 - **Copy, paste, etc.:** very useful!
 - **Data editor:** opens a rudimentary spreadsheet-like viewer for datasets that can be used to make edits. Not recommended for routine use.
 - **GUI preferences:** options to change colors, fonts, font sizes, etc., in the GUI. To make changes permanent you will need to save a new `.RConsole` file in your home directory.
- **View**
- **Misc**
 - **List objects:** lists all objects in workspace. Equivalent to command `ls()`.
 - **Remove all objects:** removes all objects from workspace. Equivalent to command `rm(list=ls(all=TRUE))`.
 - **List search path:** lists currently attached packages. Equivalent to command `search()`.
- **Packages:** functions to download and install packages.
- **Windows:** options to change how windows are displayed within GUI.
- **Help:** options and resources for getting help, including several free R manuals in PDF format.

2.1.2.2 Using R in RStudio

Although you can type R code directly into the console, or type in script windows within the base R GUI, it is usually easiest to use a separate program for writing code. The two best, in my experience, are RStudio and Notepad++ (see above). **RStudio** is an IDE designed to manage R projects, and has many useful features. Unlike some other options, RStudio has R built in so you can write code and execute it within the same program. RStudio can be used on Windows, Mac, and Linux operating systems. RStudio also has built-in support for utilities like RMarkdown and Quarto (which was used to build this website).

Once you have RStudio installed, open it and you will see a screen like this:



Your RStudio might look different at first. You can change the color scheme by going to Tools - Global Options - Appearance and selecting a theme. Here I'm using the "Classic" theme with size 12 Courier New font and 110% zoom.

RStudio has an entire environment for working with R in one window. Each of the panels performs different functions. Starting at the bottom left and going clockwise, the panels are:

- **Console:** command line interface for R. You can type commands here and execute them by pressing ENTER.
- **Script:** a space where you can type code and later execute it by either clicking "Run" or pressing CTRL-ENTER (CMD-ENTER on a Mac). Usually people write code in the script window rather than typing it directly into the console.
- **Environment:** this window lists all of the data objects that you have loaded.
- **Files:** this window lists all of the files available in your R home directory. You can also navigate to other folders on your machine and select files to open.

The bottom right panel also has a tab called "**Plots**". This is where plots that you make can be seen. Sometimes you will need to resize this panel to see the plots clearly. All of the panels can be resized by clicking and dragging the borders between them.

2.2 Commands, objects, and output

The key to understanding R is to understand [object orientation](#). This means that R code is built around manipulating objects. In programming, an **object** is a construct or thing with well-defined attributes and behaviors. For example, a graphics program might have object classes like “polygon” and “circle”. Rather than redefine “polygon” every time the user needs to draw a polygon, the program creates a polygon object with attributes like “number of sides” and “side length”. The new object is an **instance** of the polygon class. The **class** of an object defines what attributes it has and how it interacts with other objects.

2.2.1 R commands—basics

In R, *everything* is an object. We will learn more about some of the most important R object types in the examples in [Chapter 3](#). Objects are created in R by many functions.

Objects are stored to memory by **assignment**, using the left arrow `<-` (a “less than” followed by a hyphen). If an object already exists, then assignment is used to either modify the object, or replace it entirely. Here is an example:

```
a <- 3
```

This creates an object called `a`, which contains the value `3`. You can then recall the value by typing `a` in the console and pressing Enter. You can also use the value stored in `a` in calculations or functions.

```
a
```

```
[1] 3
```

```
a * 2
```

```
[1] 6
```

```
1:a
```

```
[1] 1 2 3
```

Assigning a different value to `a` will overwrite it.

```
a
```

```
[1] 3
```

```
a <- 6  
a
```

```
[1] 6
```

```
a^2
```

```
[1] 36
```

Notice that there is no “Undo” button. If you make a mistake, you need to rerun the code before your mistake to get back to where you were. Objects can be deleted by “removing them” with command `rm()`.

```
rm(a)
```

Besides assigning and working with values, R code contains comments: text that is for human readers, and is not interpreted by R. Comments are very important because they tell other people—or future you—what your code is doing and how it works. Some comments are just place markers (e.g., `# Analysis begins here`) to help you find important sections. Other comments might be more lengthy, holding explanations of why you chose to code a task the way that you did (e.g., `# condense with sapply() (maintains order of subunits)`). The latter kind of comment can be very useful to your future self.

In R, comments are made with the `#` symbol. Anything on a line after `#` is not read by R.

```
# here is a comment  
b <- 5 # here is another comment
```

You can write your code directly in the console, or in a script window. If you use the script panel in RStudio (as you should), you can send commands to the console by pressing `Ctrl-Enter` (or `Cmd-Enter` on Mac). This will send and execute the entire command that the cursor is on. In the base R GUI script window, the equivalent is `Ctrl-R` (or `Cmd-R` on Mac), but this runs only the line where the cursor rests (not the entire command).

The R console can seem daunting to new users. The `>` symbol at the start of a line is the **command prompt**. Commands must begin here. If there is a `+` at

the beginning of a line, then this means that there is an incomplete command. Press Escape to exit the incomplete command and get back to >.

When you type a command like `2+2`, the output will print to the console with a `[1]` at the start of the line. The `[1]` signifies that that line starts with the first element of the output. This is to help read outputs with lots of elements. Try the command `runif(100)` to see why.

```
2+2
```

```
[1] 4
```

```
runif(100)
```

```
[1] -0.609010534  0.347631234  2.048252306 -0.002142337 -0.835817793
[6] -0.238210107 -1.190233129 -1.610243300  0.423641941  0.720337353
[11]  0.683122862  0.781231293  0.447730149 -0.075278890  1.487092162
[16]  0.279635135  1.452459877  0.842092010 -0.770795281  0.402438917
[21] -1.039524667  1.404436428 -0.350887669 -1.091948376 -1.029570515
[26] -0.347935328 -0.778151684  0.274368621 -0.060847357  0.216199001
[31] -0.145640586  1.150290703  0.383033902  0.498943636 -0.606711509
[36]  2.537473430  0.385782226 -0.582711335  1.402764740  1.956849170
[41] -0.629295532 -0.499490281 -1.227310764  0.663030349 -0.765975502
[46]  0.858313213 -0.612187948 -0.423395969 -1.341345858 -0.167994604
[51] -0.060166914  0.113691551  0.510067944 -1.181660882 -1.211606647
[56] -0.060996503 -0.422803582  0.866378773  0.386443745 -0.498739084
[61]  2.255541651  0.916157707  0.347834578 -0.438123284 -1.044516219
[66] -0.610231133 -0.069890333  0.246013092 -0.687835550 -0.897058685
[71] -0.693152996  1.357683528  0.129845843 -0.794244828  0.556779083
[76]  0.502049081 -0.646220918 -1.286922455 -2.088388160 -0.331117138
[81]  1.127370908  1.300924386 -0.899015585 -0.972150347 -1.285693063
[86]  2.275255591 -0.589030099 -1.230740923  0.604215152 -0.146660740
[91]  0.051738808 -0.486783319  0.602872678 -1.302174473 -1.076079394
[96]  0.564145393 -0.869261540  0.324798673  0.403811992 -0.046314519
```

Commands can be split over multiple lines. When that happens, a `+` will be seen at the left side of the window instead of a `>`. A split command will be executed once a complete expression is input.

If you accidentally start a command and aren't sure how to finish it, you can exit and cancel the command with the `ESCAPE` key. R executes commands, not lines. You can have more than one command on a line separated by a semi-colon `;`, but this generally frowned upon.

```
# command split across lines:  
2 +  
1
```

```
[1] 3
```

```
# 2 commands on a line (don't do this)  
2 + 2; 3 + 3
```

```
[1] 4
```

```
[1] 6
```

Below is an example where the a command is broken across lines, with each line getting an explanatory comment. This is possible because R ignores whitespace (spaces, tabs, returns, etc.). Very complex functions or loops might benefit from this kind of commenting. Some programmers do not like this kind of comment—but you comment code the way that works for you.

```
rnorm(  
  10, # sample size  
  10, # mean  
  2   # sd  
      # note no comma after last argument!  
)#rnorm
```

```
[1] 14.689901  9.439524 11.448059 12.416139 13.202169 10.542668 10.333154  
[8]  8.076410  9.182291 12.842275
```

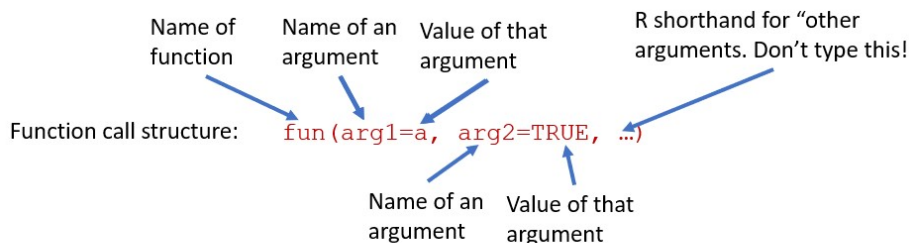
Notice in the above example that the closing parenthesis for `rnorm()` is marked with a comment. This is a good practice if the open and close parentheses are separated by many lines of code. Most IDEs will highlight matching parentheses, but marking them yourself can potentially save some headache.

When writing comments or writing multi-line commands you should keep in mind that most code editors don't do line breaks very well. Or, they will wrap text displayed in the editor, but this might not display the same on every device. As a rule I try to keep lines of code to 80 characters, so I don't have to scroll over to see the end of a command. You can do this too, by breaking up commands and long comments into multiple lines. It also helps to indent additional lines by a set amount—most people prefer either 2 or 4 spaces rather than a tab stop.

2.2.2 Using R functions

R **functions** are objects that do things to other objects, or create objects, or modify objects... basically, functions **do** stuff while most other objects **store** stuff. Functions are called by name, and they have options or **arguments** contained within parentheses. It's good practice to have the parentheses touching the function name.

Argument values are set using single equals sign `=`. Argument values must be of the correct **type** (numeric, character, logical, etc.), must take on valid values (or be one of the available options), must be spelled and capitalized correctly, and may be named or not at the user's discretion. Most functions will print their results to the console upon completion. If you want to store the results of a function, you need to assign the results to an object using `<=`.



Example function call: `rnorm(n=20, mean=10, sd=2)`

Same function call with unnamed arguments: `rnorm(20, 10, 2)`

See below for some examples of function use using `rnorm()`, which draws random numbers from a normal distribution¹. Your numbers may be different than those below, because the function is *random*.

```
# arguments named
## draw 20 values with mean 10 and sd 2
rnorm(n=20, mean=10, sd=2)
```

```
[1] 10.670469  6.913096  7.813828  7.615671  9.506108 11.683060  8.262734
[8] 15.982715  8.384820 10.311841 10.307029  7.665031  8.939590  6.610559
[15]  8.674325 13.363957 10.202968 11.923758 14.765363 10.863861
```

¹The normal distribution comes up a lot in statistics. It is sometimes called the “bell curve”, but this is inappropriate because many distributions can be bell-shaped. It is also called the “Gaussian” distribution, but Gauss did not discover it. The normal distribution is defined by its mean μ and its standard deviation (SD) σ . In other words, its central tendency and spread, respectively.

```
## equivalent with unnamed arguments
rnorm(20, 10, 2)
```

```
[1] 12.225829 13.473406 12.281592 13.116494  5.648217  7.439171 12.348314
[8]  6.096252  6.386966 10.651119  9.048785  9.466626 10.687329 13.217170
[15]  7.392369  7.836207 12.286809 13.727040 12.047255 11.372296
```

```
## very different: 2 values from a normal
## distribution with mean=20 and sd=10
rnorm(2, 20, 10)
```

```
[1] 25.80871 15.12279
```

```
## when in doubt, name your arguments!
rnorm(sd=2, n=20, mean=10)
```

```
[1]  9.879674  8.046736 11.747297 14.159690  8.834004 10.654495  6.664223
[8]  9.618570 10.043405  8.113577  8.036997  9.955106 12.871275 10.880501
[15]  8.591667 11.154480  9.172203 11.172726 13.912036  7.789074
```

You can use the R help pages to see what the arguments are, the order in which R expects them, and their default values (if any). For `rnorm()`, the mean and sd have defaults 0 and 1, respectively.

```
?rnorm
```

To recap, argument **order** is important when using functions. You can supply arguments to a function without naming them if you put them in the proper order. See the help files to see what the order is. *When in doubt, name your arguments.*

Functions expect arguments in a particular format. If a function is not working, you might have an argument formatted incorrectly. Sometimes it's helpful to look at each argument separately in the R console, outside of the function. To do this, copy/paste each argument to the R console separately and execute by hitting ENTER.

Some function arguments have default values, so you don't have to supply them. For example, `runif()`, which draws random numbers from the uniform distribution, defaults to the interval between 0 and 1. So, the two `runif()` commands below are equivalent because 0 and 1 are the defaults for the limits of the interval. The `set.seed()` command before each `runif()` command is to set the random number seed, ensuring that the results are identical. If you run the `runif()` commands without resetting the random number seed, the commands will return different results because of the random nature of the function.

```

set.seed(123)
runif(10, 0, 1)
## [1] 0.2875775 0.7883051 0.4089769 0.8830174 0.9404673 0.0455565 0.5281055
## [8] 0.8924190 0.5514350 0.4566147

set.seed(123)
runif(10)
## [1] 0.2875775 0.7883051 0.4089769 0.8830174 0.9404673 0.0455565 0.5281055
## [8] 0.8924190 0.5514350 0.4566147

```

2.2.3 The R workspace

When you use R, all objects and data that you use exist in the **R workspace**. Objects in the workspace can be called from the console. You can think of the workspace like a kind of sandbox or workbench.

Most of the time, when you start R this creates a new workspace with no objects. If you open a previously saved workspace, you will see the message **[Previously saved workspace restored]**. If you saved your R workspace without a unique name, it can be hard to figure out what workspace you are in. You can use the `ls()` command to see what is in the workspace, which might help you figure out what workspace has been loaded. If there are no objects in the workspace, the result of `ls()` will be `character(0)`. This output means that the `ls()` command returned a vector of text values (`character`) with 0 elements. In other words, the workspace contains no objects.

As you work, your workspace will fill up with objects. This isn't a big deal until you start to either have too many names to keep track of, or start to use too much memory. The first problem can be dealt with by commenting your code. The second problem can be solved by deleting unneeded objects to save space. You can check to see how much memory your R objects are using with the following command:

```

# not run, but try on your machine
sort(sapply(mget(ls()),object.size), decreasing=TRUE)

```

This command will print the sizes of objects in memory in the current workspace, in descending order. The default units are bytes. If you want the results in Mb, divide the output by 1.024e6.

```

# not run, but try on your machine
a <- sort(sapply(mget(ls()),object.size), decreasing=TRUE)
a/1.024e6

```

R objects can be deleted or removed from the workspace with the `rm()` function. You can remove objects one at a time or by supplying a set of names. Note that the set of names is supplied to an argument called `list`, which is confusing because `list` is also the name of a special type of object in R.

```
# not run, but try on your machine
x <- 1
y <- 2
z <- 3

# shows x, y, and z in workspace
ls()

# remove x
rm(x)
ls() # x is gone

# remove y and z
rm(list=c("y", "z"))
ls() # x, y, and z are all gone
```

2.3 R syntax

2.3.1 Object size and structure

As you've already seen, the most important R function is **assignment**. When you assign a value to a variable, you can then use that variable in place of the value. Assigning a value to variable name will automatically create a new object in the workspace with that name. If an object with that name already exists, assigning a new value to that name will overwrite the old object.

You should always use the left arrow `<-` as the assignment operator. The equals sign `=` can be used for assignment, but you should not do so because `=` is easily confused with the equality test operator `==`. Also, `=` is a mathematical symbol, but `<-` is unambiguously an R operation. You can also use `->` to assign to the right, but doing so usually makes your code harder to read rather than easier. Always use `<-` to assign to the left.

```
aa <- 4
bb <- 1:10
```

Don't do this:

```
cc = 5
```

Don't do this either.

```
5 -> cc
```

And definitely don't do this:

```
a <- 5 -> b
```

You can chain multiple assignments together in a single command. This is useful if you need to quickly create several objects with the same structure:

```
list3 <- list2 <- list1 <- vector("list", 5)
```

Assignment can also be used to **change or overwrite** parts of pre-existing objects. Usually this involves the bracket notation, which we will explore more in Chapter 5. In the example below, the third element of object `a`, `a[3]`, is replaced with the number 42.

```
a <- 1:5
a
## [1] 1 2 3 4 5

a[3] <- 42
a
## [1] 1 2 42 4 5
```

One little-known feature of R assignment is that it can **resize** objects. For example, if you assign a value to the *n*-th element of a vector that has fewer than *n* elements, R will increase the length of the object to the size needed. It should be noted that this can be done accidentally as well as intentionally, so you need to be very careful if you resize objects using assignment. Personally, I don't do this because it is often clearer to change the size of objects in a separate command.

```
# resize by assignment (not recommended)
a <- 1:3
a[5] <- 5
a
## [1] 1 2 3 NA 5

# resize in separate command (preferred)
```

```

a <- 1:3
length(a) <- 5
a
## [1] 1 2 3 NA NA

# resize by concatenation (preferred)
a <- 1:3
a <- c(a, 4:5)
a
## [1] 1 2 3 4 5

# alternatively:
a <- 1:3
b <- 4:5
a <- c(a, b)
a
## [1] 1 2 3 4 5

```

2.3.2 Operators

Most math operators in R are similar to those in other languages or programs. For math expressions, the standard order of operations (PEMDAS) applies, but you can use parentheses if you want to be sure. The integer sequence operator `:` comes first, and comparison tests (`==`, `<`, `<=`, `>`, and `>=`) come last. The examples below show how many common R operators function, taking advantage of the fact that you can use an object's name in place of its value in most situations.

```

# make some values to work with
aa <- 5
bb <- 8

# multiply
aa*3
## [1] 15

# add
bb+3
## [1] 11

# square root
sqrt(aa)
## [1] 2.236068

# natural logarithm (base e)

```



```

log(aa)
## [1] 1.609438

# logarithm (arbitrary base)
log(aa, base=3)
## [1] 1.464974

log(aa, pi)
## [1] 1.405954

# common logarithm (base 10)
log10(aa)
## [1] 0.69897

# exponentiation with base e (antilog)
exp(aa)
## [1] 148.4132

# Vectorized (element-wise) multiplication
bb*2:5
## [1] 16 24 32 40

# Exponentiation
bb^2
## [1] 64

# Alternate symbol for exponentiation (rare)
bb**2
## [1] 64

# Remainder (modulus)
## 13 / 5 == 2 remainder 3
13 %% 5
## [1] 3

# Quotient without remainder
13 %/% 5
## [1] 2

# Order of operations
aa+2*3
## [1] 11

# Order of operations
aa+(2*3)

```

```
## [1] 11

# Order of operations
(aa+2)*3
## [1] 21

# Sequence of integers
1:10
## [1] 1 2 3 4 5 6 7 8 9 10

# Note that ":" comes first!
1:10*10
## [1] 10 20 30 40 50 60 70 80 90 100

# Comparisons are last
2*2 > 1:5*2
## [1] TRUE FALSE FALSE FALSE FALSE
```

Some of these examples illustrate a key concept in R programming: **vectorization**. Many R functions can operate on sets of numbers, or **vectors**, just as well as on single values or **scalars**. The function will operate on every element of the vector in order. Most R functions are vectorized to some extent. This is very handy because many operations in statistics deal with vectors of values.

Vectorization can cause headaches if the vectors are not of the same length. If the length of one vector is a multiple of the length of the other, then the shorter vector gets recycled. This is sometimes called the “recycling rule”. The recycling rule can be very convenient if you know what you’re doing and are paying attention. If you don’t pay attention, or aren’t aware that recycling is happening, it can create a headache because values are recycled without telling you.

If the length of the longer vector is not a multiple of the length of the shorter vector, the shorter one will be recycled a non-integer number of times and R will return a warning (longer object length is not a multiple of shorter object length).

```
# lengths compatible:
x <- 1:2
y <- 3:6
x+y
## [1] 4 6 6 8

# lengths not compatible:
a <- 1:3
b <- 4:5
```

```
a + b
## Warning in a + b: longer object length is not a multiple of shorter object
## length
## [1] 5 7 7
```

2.4 R packages

Packages are modules that can be added to R to extend its capabilities. Most packages are written by R users and researchers who need functions that are not provided with the base R installation. These add-ons are one of the greatest strengths of R compared to other software packages. Commercial and proprietary statistics programs (like SAS) are upgraded only by a single group of developers, if at all. In contrast, new functions and capabilities are added to R by the user community every day.

Some packages are widely used and cited, others less so. Do your homework before depending on a package. Is the package included or recommended with the base R installation? Is the package associated with any peer-reviewed publications? Does the package author appear to have the credentials that would qualify them to produce a package? Do other packages depend on this package? All of these are good signs that a package is reliable.

Most packages are stored on The Comprehensive R Archive Network, or CRAN. Other repositories include R-Forge, BioConductor, and GitHub. Packages can also be downloaded and installed from local storage.

R-Forge and BioConductor can usually be accessed from the R command line just like CRAN. Installing packages from GitHub may require a different procedure. Most R packages are written in the R language itself, so even if you can't find a way to install a package you can usually just copy and run its source code.

2.4.1 Your R library

When you install R packages, the files are downloaded and stored on your machine for R to access when needed. R will probably create a folder in your R home directory for this purpose. The first time you install a package in R you will be asked if R can create a personal library in which to store the package. Package installations are specific to the version of R that you used for the installation, and specific to your operating system login. This means that if someone else logs in to your machine and wants to use a package, they will have to install the package for themselves. This also means that if you update your R installation to a new version, you may need to install packages again because some packages are version-specific.

2.4.2 Installing R packages

2.4.2.1 Installing R packages using the GUI

In the R GUI, go to Packages – Install package(s).... You will be prompted to select a CRAN mirror (i.e., server from which to download). Pick any mirror you like. Then, scroll to and select your package. If the install fails, try a different mirror by going to Packages – Set CRAN mirror..., and then retrying the installation.

2.4.2.2 Installing packages in RStudio

You can install a package in RStudio by going to Tools–Install Packages... and searching for the name of the package you want.

2.4.2.3 Installing packages using R code

In the R console, you can install packages using function `install.packages()` as shown below. You will still be asked to select your CRAN mirror (most packages are hosted on CRAN).

```
install.packages("vegan")
```

2.4.3 Working with packages in R

You can access the packages you have installed by loading them. In most situations you must load a package before you use it. The usual way is to use the `library()` function. Once a package is loaded, you can use its functions and datasets. The function `require()` does the same thing as `library()`, but it is designed for use inside other functions. Most of the time you should use `library()`. The command below will load the package `vegan` so its functions and datasets become available:

```
library(vegan)
```

```
Warning: package 'vegan' was built under R version 4.3.3
```

```
Loading required package: permute
```

```
Loading required package: lattice
```

```
Warning: package 'lattice' was built under R version 4.3.3
```

Notice that the package name does not have to be in quotation marks within the `library()` function.

You can see what packages are currently attached with `search()` or `sessionInfo()`. Packages can be unloaded detaching them from the workspace: `detach(package:x)`, where `x` is the name of the package you want to detach. In the example below, the first command loads the `MASS` package. Then, we can see that is loaded in the workspace with `search()`. The next command unloads `MASS`, then verifies that `MASS` was unloaded from the workspace with another `search()` command.

```
search()
```

```
[1] ".GlobalEnv"      "package:vegan"    "package:lattice"
[4] "package:permute" "package:stats"    "package:graphics"
[7] "package:grDevices" "package:utils"    "package:datasets"
[10] "package:methods" "Autoloads"        "package:base"
```

```
library(MASS)
search()
```

```
[1] ".GlobalEnv"      "package:MASS"     "package:vegan"
[4] "package:lattice" "package:permute"   "package:stats"
[7] "package:graphics" "package:grDevices" "package:utils"
[10] "package:datasets" "package:methods"   "Autoloads"
[13] "package:base"
```

```
detach(package:MASS)
search()
```

```
[1] ".GlobalEnv"      "package:vegan"    "package:lattice"
[4] "package:permute" "package:stats"    "package:graphics"
[7] "package:grDevices" "package:utils"    "package:datasets"
[10] "package:methods" "Autoloads"        "package:base"
```

There is rarely any need to unload a package from the workspace unless there are naming conflicts. The most common situation is when two attached packages have functions with the same name. In this situation, R will use the *most recently loaded* package's function. This phenomenon is called masking. When there is a name conflict, you can use the double colon operator `::` to specify which package's function you want to use.

Using `::` can also access a function without loading the entire package into the workspace. If both packages are already loaded into your workspace, then `::`

will simply select which package's function to use. Note that a package must be *installed* on your machine before you can use its functions.

The example below shows how `::` can be used to access the `fitdistr()` function from packages `MASS` without loading the package

```
# simple example
base::log(10)
```

```
[1] 2.302585
```

```
log(10)
```

```
[1] 2.302585
```

```
# use a function without loading its package
MASS::fitdistr(rnorm(100, 50, 10), "normal")
```

```
      mean      sd
50.6073364  9.0431206
( 0.9043121) ( 0.6394452)
```

```
# verify that MASS is not attached:
search()
```

```
[1] ".GlobalEnv"      "package:vegan"    "package:lattice"
[4] "package:permute"  "package:stats"    "package:graphics"
[7] "package:grDevices" "package:utils"    "package:datasets"
[10] "package:methods" "Autoloads"        "package:base"
```

```
sessionInfo()
```

```
R version 4.3.2 (2023-10-31 ucrt)
Platform: x86_64-w64-mingw32/x64 (64-bit)
Running under: Windows 10 x64 (build 19045)
```

```
Matrix products: default
```

```
locale:
```

```
[1] LC_COLLATE=English_United States.utf8
[2] LC_CTYPE=English_United States.utf8
[3] LC_MONETARY=English_United States.utf8
```

```

[4] LC_NUMERIC=C
[5] LC_TIME=English_United States.utf8

time zone: America/New_York
tzcode source: internal

attached base packages:
[1] stats      graphics  grDevices  utils      datasets  methods   base

other attached packages:
[1] vegan_2.6-10  lattice_0.22-7  permute_0.9-8

loaded via a namespace (and not attached):
[1] digest_0.6.33  fastmap_1.1.1  Matrix_1.6-4  mgcv_1.9-1
[5] xfun_0.52      splines_4.3.2  parallel_4.3.2  knitr_1.50
[9] htmltools_0.5.7  rmarkdown_2.29  tinytex_0.57  cli_3.6.2
[13] grid_4.3.2      compiler_4.3.2  rstudioapi_0.17.1  tools_4.3.2
[17] cluster_2.1.8.1  nlme_3.1-168    evaluate_1.0.4  yaml_2.3.8
[21] rlang_1.1.2      jsonlite_2.0.0  MASS_7.3-60

```

2.4.3.1 Package dependencies

Many R packages are built taking advantage of other R packages. The packages that a package depends on are called its dependencies or depends. Some packages have no dependencies (other than R itself), while others have many. Something to think about when deciding whether to use a package are the number and nature of its dependencies. If you use a package that depends on many other packages, then you are relying on the authors of those packages to maintain and update their code to work with new versions of R and new versions of packages. And, relying upon the authors of all of those packages' dependencies. And so on.

You can see the dependencies of a package by checking out its page on CRAN. For example, the `drc` package (Ritz et al. 2015) is [here](#). You can see the current version, what packages it depends on (“Depends”), and what packages depend on it (“Reverse depends”).

The point here is that you don’t want your code to break because one of the packages you use hasn’t been updated. This can get very frustrating if, for example, several packages you use all depend on each other, or even different versions of each other! Most R package authors and/or maintainers are pretty good about keeping their packages up to date, but some are not. If you really need a function from a package that is no longer maintained, or incompatible with your current version of R, you have three options:

1. Download and use the older version of R that is compatible with the pack-

age. This is not recommended because you will lose the benefit of recent bug fixes and modifications to R. This option is also not recommended because by rolling back to an older version of R, you will likely need to roll back to older versions of other packages.

2. Find another, current package that does what you need to do. This can be a quite a headache if you have a large codebase or a very specialized function.
3. Access the source code to the package you need, and “borrow” the functions or code you need.
4. Sometimes, if you only need 1 or 2 functions from a package, or you need to modify a function from a package to suit your needs. In these cases, you can “borrow” directly from the package source code. For most packages, the CRAN page will have a link to an online code repository (usually Github) and/or the package source as a “tarball” (“tar.gz” file). A tarball is a compressed version of the package source code that may or may not need to be compiled using special functions in R.

Because most R packages are written in R itself, you can usually find and adapt the code you need. This is almost always permitted by the license of the package because most packages are released under some version of the GNU Public License (GPL). If you do borrow code from a package, it is considered good form to attribute the package authors in some way (e.g., “we adapted code from the package”foo” version 1.0 (Nobody 2001)“).

2.4.3.2 Citing packages

Just like any software, you need to cite a package if you use it. This is both to give appropriate credit to the package authors, and to allow others to understand what you did. The appropriate citation for a package can be accessed from the console with the `citation()` command.

Citation for R itself:

```
citation()
```

To cite R in publications use:

```
R Core Team (2023). _R: A Language and Environment for Statistical  
Computing_. R Foundation for Statistical Computing, Vienna, Austria.  
<https://www.R-project.org/>.
```

A BibTeX entry for LaTeX users is


```
@Manual{,
  title = {R: A Language and Environment for Statistical Computing},
  author = {{R Core Team}},
  organization = {R Foundation for Statistical Computing},
  address = {Vienna, Austria},
  year = {2023},
  url = {https://www.R-project.org/},
}
```

We have invested a lot of time and effort in creating R, please cite it when using it for data analysis. See also 'citation("pkgname")' for citing R packages.

Citation for a package (note the quotation marks):

```
citation("MASS")
```

To cite the MASS package in publications use:

Venables, W. N. & Ripley, B. D. (2002) Modern Applied Statistics with S. Fourth Edition. Springer, New York. ISBN 0-387-95457-0

A BibTeX entry for LaTeX users is

```
@Book{,
  title = {Modern Applied Statistics with S},
  author = {W. N. Venables and B. D. Ripley},
  publisher = {Springer},
  edition = {Fourth},
  address = {New York},
  year = {2002},
  note = {ISBN 0-387-95457-0},
  url = {https://www.stats.ox.ac.uk/pub/MASS4/},
}
```

2.5 Frequently asked questions about R

2.5.1 What is R

R is both a computer program that does statistics, and the programming language for using that program. It was developed in the late 1990s and became popular among biologists by the mid-2000s. R offers several key advantages over other statistical programs:

1. It is completely free to use. All you need is a computer with the minimum hardware requirements, and a way to download it.
2. It is open source. Unlike commercial software like SAS or JMP, anyone can inspect the [source code of R](#) and see exactly how it does what it does. One can also modify R to suit their purposes, although there are better ways to add functionality.
3. R has an enormous user base, including many academic and professional statisticians and data scientists. This means that (a) new statistical methods are often implemented in R before in any other program; and (b) there are many community-driven sources of help online, such as [Stack Overflow](#).
4. R is designed explicitly for statistical analysis and data manipulation, so the learning curve is usually gentle for common use cases. In terms of difficulty, R is similar to Microsoft Excel formulas and to [SciPy](#) or [pandas](#) (which are extensions of Python, an open source language often recommended for beginners).
5. New methods and functions are constantly being added as add-on packages for R that can be downloaded for free and integrate (mostly) seamlessly with other packages. See the [CRAN task views](#) for a list of package collections by application area.

2.5.2 Why do we have to write code?

Why don't we use a point and click program? Why do we have to write code? There are 3 main reasons:

1. Code is just a way to express instructions *clearly and unambiguously* to the computer. Human language is often imprecise. Computers do exactly what you tell them to do, so you must tell them exactly what you want them to do. A programming language is a structured and orderly way of doing that.
2. Code, once written, can be *easily repeated* by running it again. Repeating a series of "point and click" instructions can be tedious, time-consuming, and highly error prone.
3. Writing the code to execute an analysis has the side effect of *documenting exactly what was done*. This can be extremely useful if there is ever a question about whether an analysis was done correctly, or how the data were handled. In R, we save our code in files called scripts that have a .R file extension.

As you learn, it's better to think of R as a language, rather than as a code. Don't think, "How do I *do* X in R?" Instead, think, "How do I *say* X in R?"

With this perspective, it becomes clear that “writing code” is really an exercise in “expressing instructions clearly and efficiently”.

2.5.3 Advantages and disadvantages

For all of its advantages, R has some drawbacks. Chief among them:

- R is free. Unlike some paid and proprietary software, there is no official help line to call if you have issues.
- Along the same lines, R is distributed with absolutely no warranty (it says so every time you open R). The accuracy of your analysis depends on the competence of the (unpaid) contributors and the user.
- R often requires more coding than SAS or other tools to get a similar amount of output or perform a similar amount of work. The **tidyverse** ecosystem of packages addresses some of these issues, but not without some costs (see Section 2.5.4).
- R is slower than SAS and other tools at handling large datasets because it holds all data and objects in memory (RAM) at once. This limits the performance of your machine if you are doing other tasks.
- R is single-threaded by default. This means that R does not handle parallel computation very easily, which could potentially speed up many tasks.
- R has a difficult learning curve because it is so programming-focused.
- Error messages that R gives are often vague and infuriating.

2.5.4 Base R vs. tidyverse

A colleague of mine who moved from using mostly SAS to using mostly R says that the tidyverse makes R feel more like SAS. The difference he was referring to is the way that SAS functions (“PROC”s, or procedures) tend to automate or abstract away a lot of the low-level functionality and options associated with their methods. Those functions and options are there, but they can be inconvenient to access. The equivalent base R functions, on the other hand, require the user to specify options and manipulate output to a far greater extent. Or, depending on your perspective, they *allow* the user to specify options and manipulate output. Some people like that, and some people don’t. The amount of interaction and coding in R require to get outputs that other programs like SAS produce automatically is, in my experience, one of the biggest stumbling blocks for new R users.

Because of the amount of work required to get base R to do anything, the degree of automation and abstraction available in the tidyverse packages (Wickham et

al. 2019) is very attractive to many R users: they can focus on high-level decisions without having to get into every picayune detail of routine workflows and standard analyses. Tidyverse packages allow for very streamlined and (once you learn it) easy to understand code. These advantages come at a cost: because the functions abstract the finer details away, those details are harder to see. This can be problematic if you need to work more directly with function inputs and outputs, or manipulate options or data in non-standard ways. For example, if a journal editor is adamant about some minute detail of formatting on a figure, it might be much easier to make the changes in base graphics rather than Tidyverse graphics. The abstraction of the low-level workings of an analysis or program can save a lot of time and headache, but can also make it harder to track down or even to notice errors (although to be fair, systematic approaches to data manipulation such as in package `dplyr` can also prevent many errors as well).

Recently Norm Matloff of the University of California, Davis posted an [essay about the advantages of base R compared to the tidyverse](#). It argues very well some ideas that I've thought for years, but could never quite articulate. Matloff is the author of several books on R, former editor of the R journal, and a longtime teacher of statistics and computer science. He argues that new R users should focus on learning base R rather than tidyverse. His main points can be summarized as follows:

- Tidyverse packages and code tend to utilize functional programming, rather than object-oriented programming. Functional programming is more abstract than object-oriented programming, which makes it harder to new users with little to no programming background.
- Tidyverse functions eschew use of the `$` operator, square brackets `[]`, and base R data structures in general². This means that people who learn only tidyverse are not equipped to take advantage of some of the most fundamental and powerful features of R—including many statistical functions!
- Tidyverse code avoids loops, a fundamental programming construct. Loops are, for beginning programmers, easy to “step through” and debug. In tidyverse, operations that would be in loops in base R are “hidden” inside functions. This makes it very hard to track down errors. There are base R functions that can be used to avoid loops as well (e.g., `lapply()` and its cousins), and they are equally hard for beginners to use.
- Tidyverse is intentionally designed to equip users with a small set of R tools. This means that tidyverse-only users are less equipped to deal with new situations.

²In my humble opinion, this aspect of tidyverse is just insane.

In my own workflow, and in this course, I tend to use base R methods instead of tidyverse for reasons above, as well as a few other reasons that are entirely personal and subjective:

- I learned R before tidyverse became a thing. This isn't an advantage or disadvantage to either paradigm; it just is. For me, the time and effort costs of switching to tidyverse would likely outweigh the benefits.
- I prefer the syntax of base R to the “grammar” of tidyverse. In particular, tidyverse makes extensive use of the pipe operator `%>%`, which tends to make for obfuscated (to me) code. In base R, multi-step operations are laid out on multiple lines, with intermediate objects being available in the workspace. This is not the case in tidyverse.
- I prefer to write code with as few dependencies as possible. This makes it less likely that some random update will break my code.

The other two reasons are more functional and less subjective:

- Debugging and error correction is easier in base R because of the lack of abstraction. If every step is coded manually and out in the open, errors are easier to find, isolate, and fix.
- Debugging and fixing tidyverse code often requires knowing base R, but the reverse is never true.

The takehome message is that while both base R and tidyverse offer powerful tools for working with and analyzing data, they represent two overlapping but different programming paradigms. Which framework you use depends on what level of abstraction vs. explicitness you are comfortable with in your own code. Base R makes it easier to work flexibly with the different kinds of inputs and outputs, but usually at the cost of a steeper learning curve. tidyverse makes routine tasks more streamlined with a unified syntax and grammar for data manipulation, but at the cost of flexibility. My advice is to learn base R first, then move on to tidyverse if and only if it has a specific tool you need that is not available in base R.

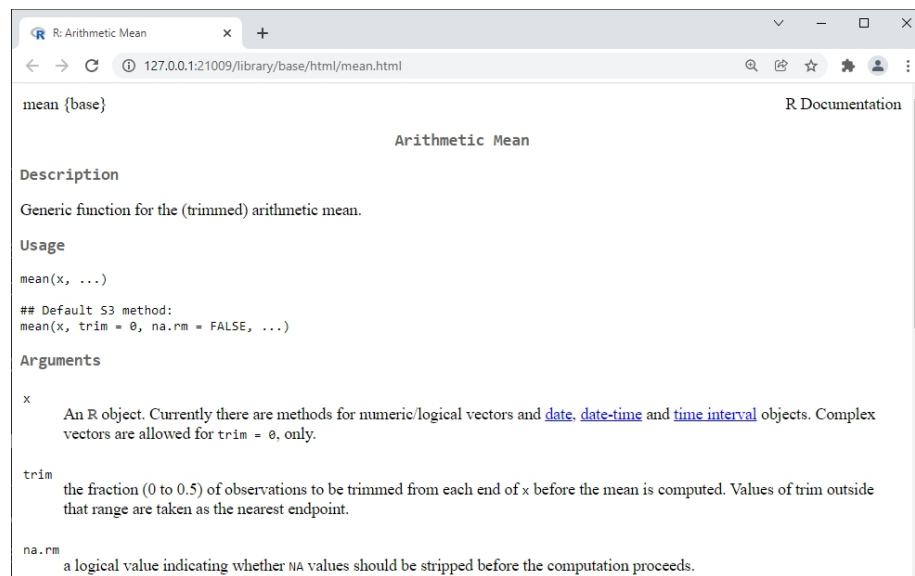
2.5.5 Where can I get R help?

The most important message of this course is simple: ***Don't panic!*** Lots of people have trouble with R. The learning curve is steep, and the error messages are inscrutable. But, there are a lot of resources available to help you.

2.5.5.1 R documentation (help) files

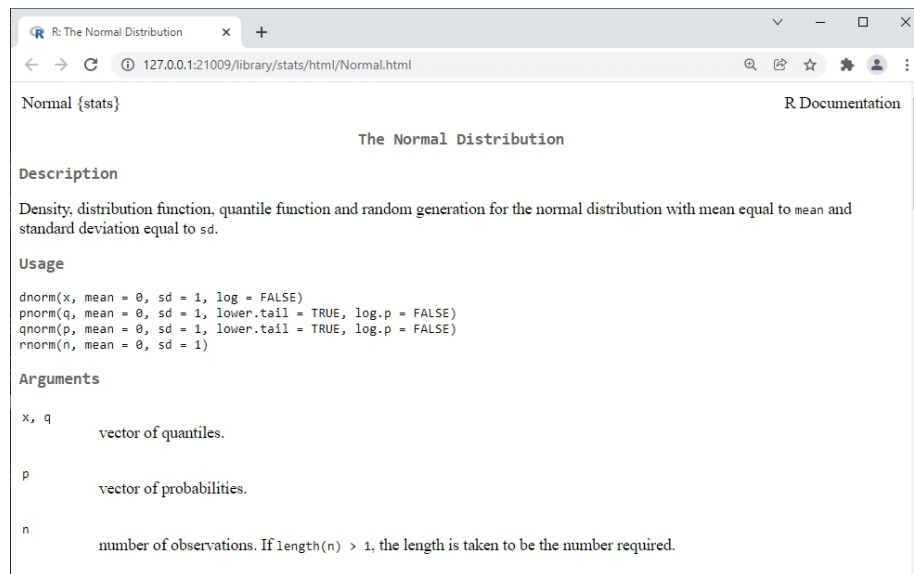
Documentation for each function, or help files, come included with the base R installation. You can access them from within R by using the `?` command. `?functionname` will open the documentation file for function `functionname`. `?` will search all currently attached packages. The command below opens the page for the `mean` function. Notice that this file is stored on your local machine, so no internet access is required.

```
?mean
```



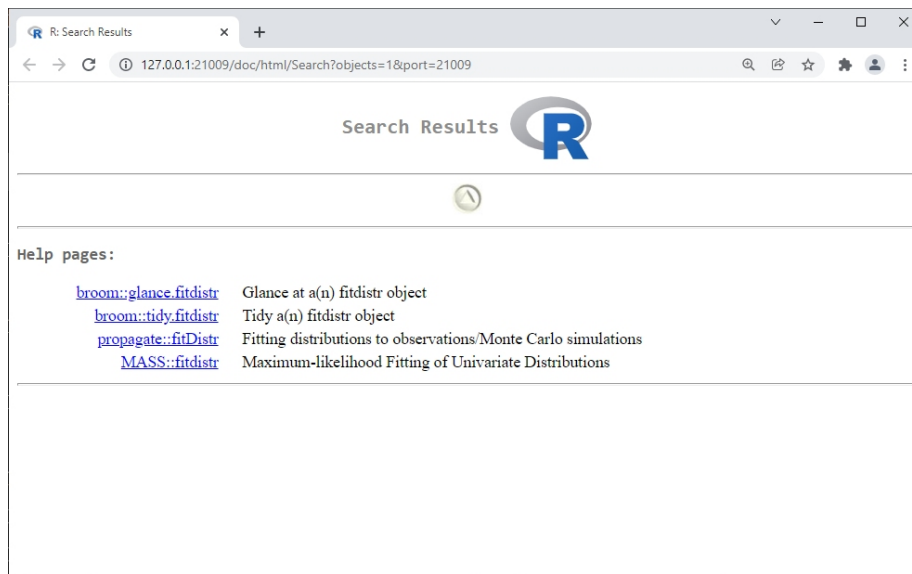
Some functions belong to a family of related functions that share a common documentation page. For example, all of the functions related to the normal distribution share a common page:

```
?dnorm  
?pnorm  
?qnorm  
?rnorm
```



You can also search for help on functions in packages that are not attached. Or, you might want to know about a function but don't know what package it belongs to. Either way, `??functionname` is a more general search option that will search any package in your library. Compare what happens if search for function `fitdistr()` using `?` and `??` (this function is in package `MASS`, and so not loaded by default). With `??`, R searches all installed packages and returns a browser window with every result that might match your search term.

```
?fitdistr
??fitdistr
```



The R documentation pages all tend to follow the same format:

Description: brief description of what the function does.

Usage: presentation of the function with its arguments in their typical format and expected order.

Arguments: list of the arguments to the function in order, their required type/format, and allowed values. This section is very important. When my code isn't working, the answer to why is usually in the arguments section.

Details: additional information necessary to use the function.

Value: description of the function's outputs.

Source: description of where the original algorithm or method came from.

References: citations for the methods used.

See also: list of similar or related R functions.

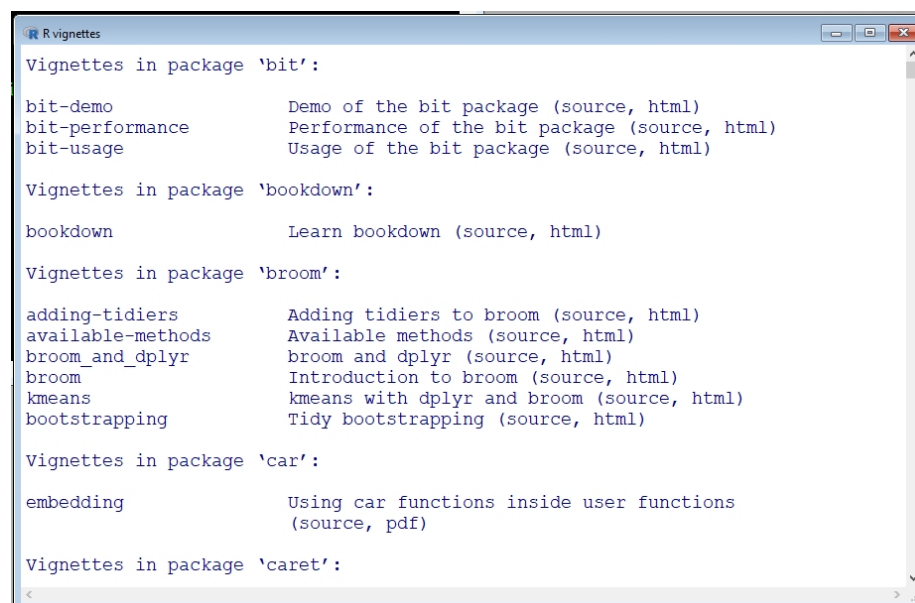
Examples: illustration of how to use the function. This section is *very* important. If your code is not working, compare how you are using the function to the examples. Are you inputs in the right format? Did you name your arguments or put unnamed arguments in the wrong order? Careful examination and running of the examples will usually reveal why your code is not working.

R documentation pages tend to be very terse and include very little in the way of background or explanation. They are generally meant to refresh the memory of people already familiar with R and with statistics, not to educate new R users or inexperienced statisticians. If you fall into one of the latter categories, or if you are trying to learn a new function, read on.

2.5.5.2 R vignettes

A **vignette** is a document demonstrating the use of an R package or its functions, with more details and explanation than in the help files. Not all packages offer vignettes. You can see what vignettes are available with the command `vignette()`. Specific vignettes are accessed by name and package. The examples below show how to get a list of available vignettes, available vignettes by package, and open specific vignettes.

```
# list vignettes in all installed packages
vignette()
```



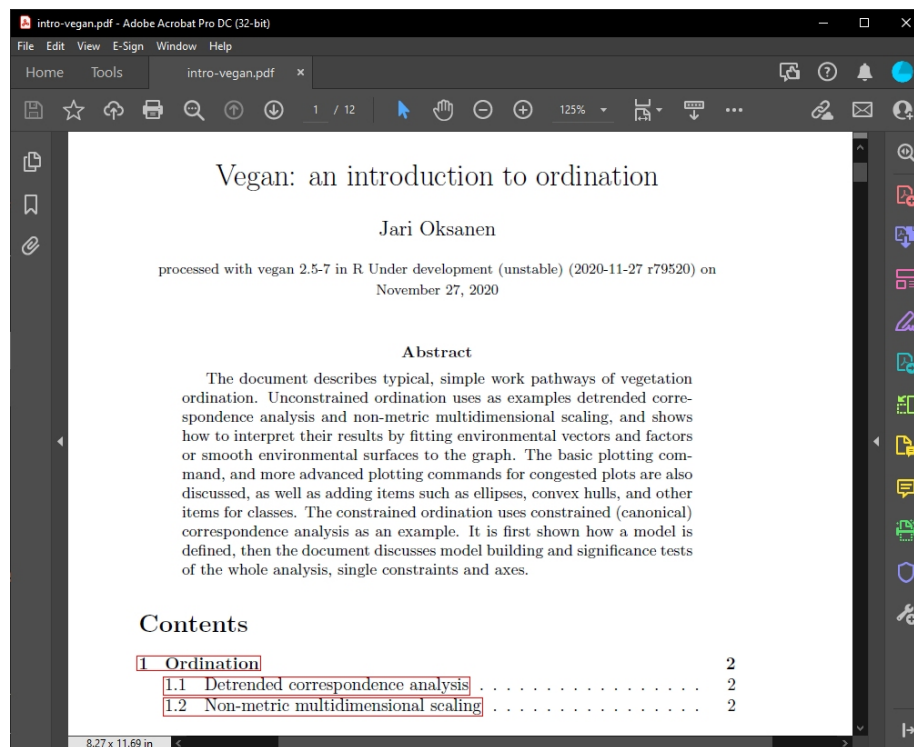
```
vignette(package="vegan")
```

```
R vignettes

Vignettes in package 'vegan':

decision-vegan      Design decisions and implementation (source, pdf)
diversity-vegan     Diversity analysis in vegan (source, pdf)
intro-vegan         Introduction to ordination in vegan (source, pdf)
partitioning        Partition of Variation (source, pdf)
FAQ-vegan           vegan FAQ (source, html)
```

```
# open vignette "intro-vegan" in local PDF reader
vignette("intro-vegan", package="vegan")
```



2.5.5.3 Official resources

The R project maintains an [official help page](#) with lists of places to get help. They also maintain a [list of manuals and documentation](#). If you want a handy

desk reference, you could do worse than the [R reference card](#) (accessed 2021-12-21).

The [Journal of Statistical Software](#) publishes papers about new statistical algorithms and software packages. Many of its papers are about new R packages and applications. The papers about new R packages and methods often include extensive code examples.

2.5.5.4 R task views

The [Task Views](#) on CRAN contain frequently updated lists of packages that are useful for specific domains. For example, ecologists might check the [Environmetrics](#), [Hydrology](#) or [Spatial](#) task views. Paleontologists might be more interested in [Paleontology](#) or [Phylogenetics](#). Physiologists and biochemists might be interested in [Pharmacokinetics](#), [Clinical Trials](#), or [Survival](#). I've spent a lot of time in the [Bayesian](#), [Numerical Mathematics](#), and [Machine Learning](#) task views. There's something for everyone...even a [Teaching Statistics](#) task view with resources to help you learn R and stats at the same time.

(All links accessed 2025-08-06).

2.5.5.5 R mailing lists

If your question has been asked before—and this is very likely—there may be an answer waiting for you on the [R Help mailing list](#). Results from the R help mailing list will usually show up in Google results. You can submit a question to the R mailing list, but be very cautious about doing so. You will need to provide a short, clear, reproducible example in your question so that other people can see the issue you are having³. You also need to be sure that your question hasn't been asked before. Some members of the mailing list will respond with great hostility and condescension if you fail to follow those rules.

There are several [special interest groups \(SIG\)](#), e-mail mailing lists for R users who work in specific fields. For example, [R-sig-ecology](#) is an e-mail list for ecologists who use R. Other SIG of interest to biologists include [R-sig-genetics](#) for genetics, [R-sig-mixed-models](#) for mixed models, [R-sig-phylo](#) for phylogenetics and related methods, and [R-sig-Geo](#) for GIS and spatial analysis.

³This means a *very* minimal code example that people can use to get the same problem that you are having. Don't post your entire dataset or include many lines of pre-processing steps or other code that led up to your problem. Use base R functions or built-in datasets to create a small dataset or set of values that will immediately provoke the same error code. Often the process of doing this will help you solve your own problem!

2.5.5.6 Unofficial online resources

Unofficial resources are becoming more and more abundant and useful. One of the most important sources for R help is [StackOverflow](#). StackOverflow is a hugely popular forum site where programmers ask and answer questions, and has an active [R section](#). When (not if) you find a solution on StackOverflow, include a link to the answer in your code as a comment. This can save you some headache down the line when you need to remember how you solved a problem. Other useful resources include [RSeek.org](#), a search engine for R resources; and [R-Bloggers](#), which aggregates many blogs about R and statistics.

Many R users maintain blogs or websites that may or may not be aggregated on R-Bloggers. You can usually find these via a Google search. I have used the pages on [Quick-R](#) many times. The UCLA Statistical Consulting Group has an [excellent collection of tutorials](#).

2.5.5.7 R books

The last few years have seen an explosion of books published specifically about the R language and its uses. Some of these books are general introductions to R and statistics; others are very domain specific. Inclusion of a book here does not constitute specific endorsement. I own some of these, but others are included because they are common R-related references that you might find helpful.

- The [Use R! series](#) contains many high-quality general and domain-specific books, although some of the older titles are a bit dated. For example:
 - A Beginner’s Guide to R by Alain Zuur et al.
 - Data Manipulation with R by Phil Spector
 - Introductory time series with R by P.S.P. Cowperthwaite and A.V. Metcalfe.
 - Bayesian computation with R by Jim Albert
 - Introductory Statistics with R by Peter Dalgaard
 - Morphometrics with R by Julien Claude
- The R Book by Michael Crawley
- Advanced R by Hadley Wickham
- R for Dummies by Andrie DeVries

When I was first starting out with R I found several books invaluable. Having code examples printed where I could highlight, annotate, etc., was really helpful. Over time I’ve found myself relying less on textbooks and more on online sources (especially StackOverflow). Still, there are a few books that I use frequently, for their statistical exposition more than their code examples. At least one of Bolker (2008), Kéry (2010), Legendre and Legendre (2012), A. F. Zuur, Ieno, and Smith (2007), and A. F. Zuur et al. (2009) is usually on my desk at any given time.

2.5.5.8 Two reminders

I said it at the beginning of this section, and it bears repeating: ***Don't panic!*** R has a steep learning curve, but the pay-off is worth it. There are plenty of resources and people out there who can help you.

Finally, be sure to ***read the manual*** when you are stuck. The R help pages and code examples usually have a solution. Make sure your inputs are in the right format. Check for syntax and spelling error—including capitalization! As a funny meme on I saw on Reddit said, *6 hours of debugging can save you 5 minutes of reading documentation.*

Chapter 3

R data types and objects

3.1 R data types

In R, data are stored in **objects** with a well-defined structure. The structures of these objects can facilitate manipulating and retrieving your data. The structure of an R object is determined by its **class**. The format of data (i.e., the kinds of values contained in an object) is referred to as the data **type**. Most R users only encounter types when trying to understand error messages. This is because many R functions will return errors if their inputs are not of the correct type.

3.1.1 Character type

Character data are text strings. They are input and printed using quotation marks (""). Note that straight quotes are used. No other kind of quotation mark is valid in R code. If you paste in text from another program that has “curly quotes” (e.g., from Word), you will get an error.

```
a <- c("aardvark", "bunny", "caterpillar")  
a
```

```
[1] "aardvark"    "bunny"       "caterpillar"
```

Notice that in the command above the values in **a** are defined in quotation marks. Without quotation marks, R would interpret those words as the names of objects. For example:

```
caterpillar <- "lion"
c("aardvark", "bunny", caterpillar)
```

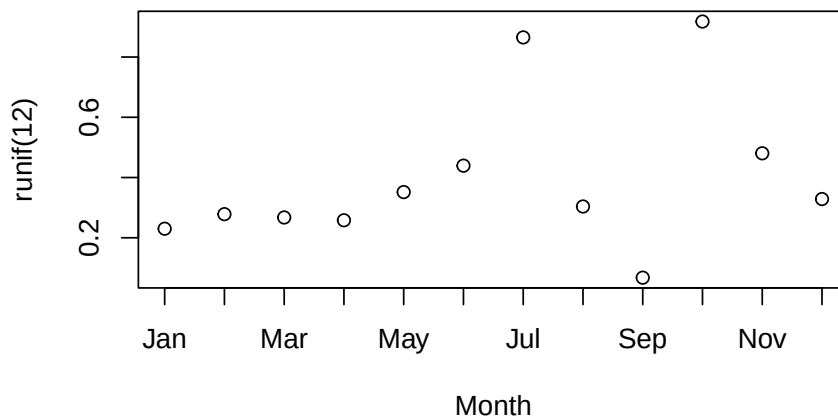
```
[1] "aardvark" "bunny"    "lion"
```

There are some special character vectors pre-built into R that can be useful in programming: `letters` returns the lower case letters a-z, while `LETTERS` returns the upper-case letters A-Z.

```
letters
## [1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q" "r" "s"
## [20] "t" "u" "v" "w" "x" "y" "z"
LETTERS
## [1] "A" "B" "C" "D" "E" "F" "G" "H" "I" "J" "K" "L" "M" "N" "O" "P" "Q" "R" "S"
## [20] "T" "U" "V" "W" "X" "Y" "Z"
letters[11:20]
## [1] "k" "l" "m" "n" "o" "p" "q" "r" "s" "t"
```

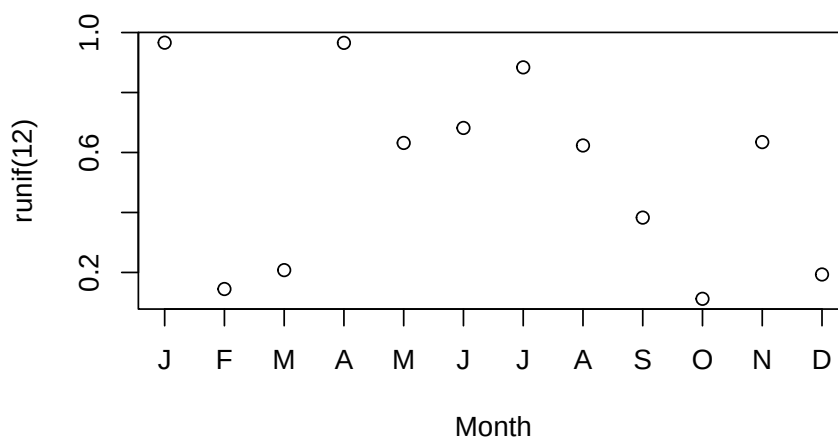
Similarly, `month.name` and `month.abb` contain the names of the 12 months and their 3 letter abbreviations, respectively. The latter is very useful for making figure axes.

```
plot(1:12, runif(12), xaxt="n", xlab= "Month")
axis(side=1, at=1:12, labels=month.abb)
```



As an aside, the function `substr()` extracts parts of character strings (“substring”). One application is getting one letter abbreviations of the months for a figure axis. The example below selects from the month names starting and ending at character 1. The argument `xaxt="n"` in `plot()` suppresses the x -axis, so we can add our own with `axis()`.

```
use.months <- substr(month.name, 1, 1)
plot(1:12, runif(12), xaxt="n", xlab= "Month")
axis(side=1, at=1:12, labels=use.months)
```



3.1.2 Numeric type

Most data are stored as the **numeric** type. This includes **floating point numbers** (e.g., 3.14159 or 3.14e0). Floating point numbers are referred to as type “double” in R (short for “double-precision floating point”).

```
typeof(2)
```

```
[1] "double"
```

```
typeof(pi)
```

```
[1] "double"
```


3.1.3 Integer type

Integers are numbers with no fractional components. Some numbers are stored as integers, but integers are not always interchangeable with floating point numbers. For example, some probability distribution functions return integer values rather than floating point values (e.g., binomial or Poisson). This can be problematic if very large numbers are expected because the maximum integer value in R is much smaller than the maximum floating-point value. Note that having the type `integer` in R and *being an integer* from a mathematical standpoint are not synonymous.

```
# numbers are double by default:
typeof(7)
## [1] "double"

# convert to integer with as.integer():
typeof(as.integer(7))
## [1] "integer"

# test whether input is integer type:
is.integer(7)
## [1] FALSE

# test whether input is numeric type:
is.numeric(7)
## [1] TRUE

# some functions return integer type as output:
typeof(rbinom(1, 10, 0.7))
## [1] "integer"

# 7 is mathematically an integer (7 mod 1 == 0),
# but is not of integer type in R
7 %% 1
## [1] 0
```

Integers are used as indices for subsetting and accessing other objects. A sequence of integers can be made using the colon operator as follows:

```
a <- 1:10
a
## [1] 1 2 3 4 5 6 7 8 9 10
typeof(a)
## [1] "integer"
```

An important consequence of using integers as index values is that object dimensions are limited to the **R integer limit**. This is probably not a major concern for most users. Your machine will likely run out of memory before you could make such an object anyway. You can check the integer limit on your machine with this line:

```
.Machine$integer.max
```

```
[1] 2147483647
```

On my machine, this is about 2.14×10^9 . If you need data objects with dimensions larger than this, you may need to either use a specialized R package that can handle huge objects, or you may need rethink what you are doing.

3.1.4 Logical type

The **logical** data type stores TRUE and FALSE values. Note that TRUE and FALSE are always written in capital letters, without quotation marks. Logical values are produced by logical tests, such as the test for equality using ==:

```
1+1 == 2
## [1] TRUE
typeof(1+1 == 2)
## [1] "logical"
typeof(1+1 == 3)
## [1] "logical"
```

Another useful test you can do is to test whether elements of one set are in another:

```
5 %in% 1:10
## [1] TRUE
5 %in% 6:10
## [1] FALSE
1:10 %in% 5:15
## [1] FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE TRUE
```

You can test with inequalities:

```
2 >= 1
## [1] TRUE
3 < 2
## [1] FALSE
```

Sometimes it can be useful to store logical values as 1 or 0. You can do this by multiplying a logical by 1 or using function `as.numeric`.

```
1*(1+1 == 2)
## [1] 1
1*(1+1 == 3)
## [1] 0
typeof(1+1 == 2)
## [1] "logical"
typeof(1*(1+1 == 2))
## [1] "double"
typeof(as.numeric(1+1 == 1:5))
## [1] "double"
```

The `!` symbol **negates** or **reverses** logical values. It does not negate logical values stored as numbers.

```
2+2 == 4
## [1] TRUE
! 2+2 == 4
## [1] FALSE
1:10 %in% 5:15
## [1] FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE TRUE
!1:10 %in% 5:15
## [1] TRUE TRUE TRUE TRUE FALSE FALSE FALSE FALSE FALSE FALSE
```

The `any()` and `all()` functions test to see if any values in a set of logical values is true (`any()`) or if all of them are true (`all()`).

```
1:5 < 3
## [1] TRUE TRUE FALSE FALSE FALSE
any(1:5 < 3)
## [1] TRUE
all(1:5 < 3)
## [1] FALSE
```

TRUE and FALSE can also be abbreviated as uppercase T or F, but not lowercase `t` or `f`. This means that you should **not** create objects with names `T`, `F`, `TRUE`, etc. (don't name an object `t` either, because that is the name of the transpose function). The code block below illustrates the mischief that can be created by naming an object `T` and masking the built-in value for true.

```
# (1+1==2) is true.
T == (1+1==2)
## [1] TRUE
```

```

# make a new object T with value 3
T <- 3

# (1+1==2) is now not true, unless it is
T == (1+1==2)
## [1] FALSE
TRUE == (1+1==2)
## [1] TRUE

# remove the troublesome T
rm(T)

# things are back to normal
T == (1+1==2)
## [1] TRUE

```

3.1.5 Special values

3.1.5.1 Missing values NA

Missing data in R are labeled as NA, or “Not Available”. This is R-speak for a blank or missing value. Some functions can ignore NA, some cannot. Some functions produce NA when you supply them with improper inputs. Note that this is a feature, not a bug. R is set up to produce NA by default in many cases to help you track down where the missing values are coming from. Function `is.na()` tests to see if a value is missing.

```

aa <- c(1:8, NA)

aa + 2
## [1] 3 4 5 6 7 8 9 10 NA

# returns NA
mean(aa)
## [1] NA

# option to ignore NA values
mean(aa, na.rm=TRUE)
## [1] 4.5

# test for NA
is.na(aa)
## [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE

```

```

# which elements are NA?
which(is.na(aa))
## [1] 9

# remove NA elements
aa[-which(is.na(aa))]
## [1] 1 2 3 4 5 6 7 8

# another way to remove NA elements
aa[which(!is.na(aa))]
## [1] 1 2 3 4 5 6 7 8

# returns NA: Poisson distribution supported only on [0,+Inf]
rpois(10,-2)
## Warning in rpois(10, -2): NAs produced
## [1] NA NA NA NA NA NA NA NA NA NA

```

3.1.5.2 Infinities

Positive and negative infinity ($+\infty$ and $-\infty$) are labeled in R as **Inf** and **-Inf**. Dividing a non-zero number by 0 produces **Inf**. Taking the log of 0 produces **-Inf**. Trying to generate a value greater than R's maximal value can produce **Inf**.

```

5/0
## [1] Inf
log(0)
## [1] -Inf
10^10^10
## [1] Inf

```

3.1.5.3 Undefined values

Some values that are undefined are labeled **NaN** (not a number). This means that a value is undefined, not that it is missing (which would be **NA**) or infinite (which would be **Inf** or **-Inf**). Dividing 0 by 0, or taking the log of a negative number, produces **NaN**. The function **is.nan()** tests to see if a value is **NaN**.

```

0/0
## [1] NaN
log(-1)
## Warning in log(-1): NaNs produced
## [1] NaN

```

3.1.5.4 Special values are useful

Why are there so many special values? Often they are meant to help track down where erroneous values are coming from. If every undefined value returned the same symbol (or word, etc.), then it would be harder to know why a program was returning undefined values. Instead, the different results help point the way:

- **Inf**: Positive non-zero number divided by 0
- **-Inf**: Negative non-zero number divided by 0 *or* $\log(0)$.
- **NaN**: 0/0 or logarithm of a negative number

3.2 Basic R data structures

In R, data are stored in **objects** with a well-defined structure. The structures of these objects can facilitate manipulating and retrieving your data. The structure of an R object is determined by its class. The format of data (i.e., the kinds of values contained in an object) is referred to as the data type.

For a solid introduction to R classes and data types, refer to [An introduction to R](#), one of the official R help manuals. The book *Data manipulation with R* (Spector 2008) is also good. If you want to explore the odder and less benign aspects of R data structures, I recommend the truly excellent *The R Inferno* (accessed 2025-08-06).

3.2.1 Vectors

In R, as in mathematics, a **vector** is an ordered set of values. Vectors are a key concept in statistics and linear algebra. For example, the set $[1, 2, 3, 4, 5]$ is a vector. This vector is different than the set (and vector) $[2, 3, 1, 5, 4]$. These sets are different vectors because even though they contain the same values, those values are in different orders. In R we could call either of those sets a “vector”. Note that we don’t call them “lists” because a list is a special kind of object in R that is not synonymous with vector.

As a language designed primarily for statistics, R has a strong emphasis on vectors and their manipulation. Vectors are probably the simplest type of data object in R. Understanding them is the key to understanding other data objects in R.

Another way to think of vectors is as an **array** with one dimension, length. In computing, an “array” is a systematic arrangement of similar elements. An array with 1 dimension is a vector. A two-dimensional array is a **matrix**, and an array with ≥ 3 dimensions is just called an array. Vectors, matrices, and

arrays in R can be a convenient way to store structured data that are all of the same type (numeric, logical, character, etc.).

Vectors can be created by the function `vector()` as well as by functions specific to the type of value to be stored. The most common types are **numeric**, **logical**, and **character**. These hold numbers, logical values, and text, respectively. Each type has a default value (0, `FALSE`, and "" (blank), respectively). Notice that in each command below R prints the results to the console because the resulting vectors are not assigned to an object.

```
vector("numeric", 5)
## [1] 0 0 0 0 0
vector("logical", 5)
## [1] FALSE FALSE FALSE FALSE FALSE
vector("character", 5)
## [1] "" "" "" "" ""

# equivalent to above commands:
numeric(5)
## [1] 0 0 0 0 0
logical(5)
## [1] FALSE FALSE FALSE FALSE FALSE
character(5)
## [1] "" "" "" "" ""
```

Creating vectors this way is cumbersome, but useful when programming complex tasks. It is usually more efficient, from a memory usage standpoint, to create an object and fill it later than to create it and repeatedly change its size.

You can also create vectors by defining their contents. When you do this, R will guess at the type of vector based on the values you give it. These guesses are usually okay.

```
# numeric:
class(runif(10))
## [1] "numeric"

# integer, which is stored in numeric vectors
# despite being a different type:
class(1:5)
## [1] "integer"
is.numeric(1:5)
## [1] TRUE
```

Vectors can also be made by putting values together using the function `c()` (short for “combine” or “concatenate”). Note that if you try to combine values of different types, R will convert all values to character strings.

```

# create a vector of numbers
my.vec <- c(1,2,4:8,10)
my.vec
## [1] 1 2 4 5 6 7 8 10

# create a vector of character strings
my.vec2 <- c("zebra","hippo","lion","rhino","rhino")
my.vec2
## [1] "zebra" "hippo" "lion" "rhino" "rhino"

# create a vector with 1 numeric, 1 character,
# and 1 logical value...all are converted to
# character strings.
my.vec3 <- c(1,"hippo",1+1==3)
my.vec3
## [1] "1" "hippo" "FALSE"

```

Each value within a vector is called an **element**. Elements of a vector are accessed by the bracket notation.

```

# get some random numbers
a <- runif(5)

# print to console
a
## [1] 0.1109758888 0.1019566793 0.5136246532 0.3913545532 0.0005385461

# 3rd element
a[3]
## [1] 0.5136247

# elements 2-4
a[2:4]
## [1] 0.1019567 0.5136247 0.3913546

# elements other than 1
a[-1]
## [1] 0.1019566793 0.5136246532 0.3913545532 0.0005385461

# elements other than 1 and 2
a[-c(1:2)]
## [1] 0.5136246532 0.3913545532 0.0005385461

```

You can inspect a vector (or any object) by printing it to the console. Do this by typing the object's name into the console at the command prompt > and

pressing ENTER.

```
my.vec
```

```
[1] 1 2 4 5 6 7 8 10
```

The [1] at the start of the line with your results indicates that that line starts with the first element of the object. If an object runs over to multiple lines, each line will begin with the index of the first value printed on that line. Make your console window narrower and try this command:

```
runif(20)
```

```
[1] 0.1322838 0.8368780 0.3034587 0.4214264 0.3137365 0.6710904 0.7331663
[8] 0.6987783 0.1380413 0.8094151 0.4218067 0.1963280 0.8513804 0.3244144
[15] 0.9054958 0.2878871 0.9647381 0.2260411 0.1383193 0.2943215
```

Each element of a vector can have a name, set or accessed using the function `names()`. Element names can even be used to access elements of a vector.

```
a <- runif(3)
names(a) <- c("value1", "value2", "value3")
names(a)
## [1] "value1" "value2" "value3"
a
## value1 value2 value3
## 0.1409563 0.1493537 0.3478216
a["value3"]
## value3
## 0.3478216
```

Vectors can be extended using `c()` or by changing their length. Changing the length of a vector can also shorten it. If a vector is made longer than the number of values it has, the extra elements will be filled in with `NA`, which is the label R uses for blanks.

```
# make a numeric vector of length 8
b <- 1:8

# combine it with another value to
# increase length to 9
b <- c(b, 10)
b
```

```
## [1] 1 2 3 4 5 6 7 8 10

# remake our vector with length 8
b <- 1:8

# set its length to 10
length(b) <- 10
# gaps filled with NA
b
## [1] 1 2 3 4 5 6 7 8 NA NA

# remake our vector with length 8
b <- 1:8
# change length to 6 (shorter)
length(b) <- 6
# vector truncated
b
## [1] 1 2 3 4 5 6
```

As mentioned elsewhere, many R functions take vectors as input and use them one element at a time. This is very handy when working with data, because you will often need to perform the same operation on every observation. Like any R data object, you can use a vector in a function simply by using the name in place of the values:

```
my.vec <- 1:8
my.vec*2
## [1] 2 4 6 8 10 12 14 16
mean(my.vec)
## [1] 4.5
dpois(my.vec, lambda=5)
## [1] 0.03368973 0.08422434 0.14037390 0.17546737 0.17546737 0.14622281 0.10444486
## [8] 0.06527804
```

3.2.1.1 Atomic vectors

A final note about vectors: some R functions, help pages, and error messages refer to “atomic vectors”. These are any object type whose elements cannot be split (i.e., “atomic” in the ~~pedantic~~original sense). For example, the scalar 2 is considered atomic, being a numeric vector of length 1 whose sole element cannot be split up into something smaller. Likewise, the vector 2:3 is also atomic, because neither of its elements (2 and 3) can be split up into small elements. This definition extends to arrays with ≥ 3 dimensions.

```
a <- 4
a
## [1] 4
is.atomic(a)
## [1] TRUE
```

```
a <- runif(10)
a
## [1] 0.66896082 0.11511300 0.43171935 0.74580225 0.02561873 0.39903334
## [7] 0.96399060 0.62007693 0.58486166 0.05853039
is.atomic(a)
## [1] TRUE
```

```
a <- matrix(1:6, ncol=3)
a
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
is.atomic(a)
## [1] TRUE
```

```
a <- array(1:12, dim=c(3,2,2))
a
## , , 1
##
##      [,1] [,2]
## [1,]    1    4
## [2,]    2    5
## [3,]    3    6
##
## , , 2
##
##      [,1] [,2]
## [1,]    7   10
## [2,]    8   11
## [3,]    9   12
is.atomic(a)
## [1] TRUE
```

On the other hand, a **list** is not atomic but rather **recursive**. This means that any of its elements can be another list. Consider the following:

```
a <- list(x=1:12, y="a", z=list(m=t.test(rnorm(20)), n="a"))
a
```

```
## $x
## [1] 1 2 3 4 5 6 7 8 9 10 11 12
##
## $y
## [1] "a"
##
## $z
## $z$m
##
## One Sample t-test
##
## data: rnorm(20)
## t = -0.49708, df = 19, p-value = 0.6248
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
## -0.5846949 0.3602697
## sample estimates:
## mean of x
## -0.1122126
##
##
## $z$n
## [1] "a"
```

```
is.atomic(a)
## [1] FALSE
is.recursive(a)
## [1] TRUE
```

The list `a` is recursive because any of its elements could be another list. However, consider the list `b`:

```
b <- list(x=1, y=2, z=3)
b
## $x
## [1] 1
##
## $y
## [1] 2
##
## $z
## [1] 3
```

```
is.atomic(b)
## [1] FALSE
is.recursive(b)
## [1] TRUE
```

Note that while every element of `b` is atomic in and of itself (satisfying the definition of an atomic object above), `b` is in fact recursive. This is because any element of `b`, *could* be a list like `b`. This is a bit paradoxical, but not enough to cause any trouble.

Why do I bring this up? Because it is very easy to provoke the dreaded, `operator invalid for atomic vectors` error. This usually means you are trying to treat a non-list as a list.

```
# make a simple matrix
a <- matrix(1:4, nrow=2)
colnames(a) <- c("a", "b")
a
```

```
      a b
[1,] 1 3
[2,] 2 4
```

```
# not run, but try on your machine
```

```
# incorrect: using $ to extract from an atomic vector
# (in this case a matrix)
a$a
```

```
# correct: using column names and []
a[, "a"]
```

```
[1] 1 2
```

Atomic vectors can contain 1 and only one type of data: logical, integer, double, character, complex, and raw.

3.2.2 Data frames

Most of the time when you work with data in R, you will work with objects of the **data frame** class. A data frame looks and acts a lot like a spreadsheet. One key aspect of data frames is that data frames can store data of more than

one type, while vectors, matrices, and arrays cannot. The examples below use the built-in data frame `iris`.

Each **row** of a data frame usually corresponds to one **observation**. Each **column** contains the values of one **variable**. You can access data frame columns by name with the `$` operator:

```
iris$Sepal.Length
```

```
[1] 5.1 4.9 4.7 4.6 5.0 5.4 4.6 5.0 4.4 4.9 5.4 4.8 4.8 4.3 5.8 5.7 5.4 5.1
[19] 5.7 5.1 5.4 5.1 4.6 5.1 4.8 5.0 5.0 5.2 5.2 4.7 4.8 5.4 5.2 5.5 4.9 5.0
[37] 5.5 4.9 4.4 5.1 5.0 4.5 4.4 5.0 5.1 4.8 5.1 4.6 5.3 5.0 7.0 6.4 6.9 5.5
[55] 6.5 5.7 6.3 4.9 6.6 5.2 5.0 5.9 6.0 6.1 5.6 6.7 5.6 5.8 6.2 5.6 5.9 6.1
[73] 6.3 6.1 6.4 6.6 6.8 6.7 6.0 5.7 5.5 5.5 5.8 6.0 5.4 6.0 6.7 6.3 5.6 5.5
[91] 5.5 6.1 5.8 5.0 5.6 5.7 5.7 6.2 5.1 5.7 6.3 5.8 7.1 6.3 6.5 7.6 4.9 7.3
[109] 6.7 7.2 6.5 6.4 6.8 5.7 5.8 6.4 6.5 7.7 7.7 6.0 6.9 5.6 7.7 6.3 6.7 7.2
[127] 6.2 6.1 6.4 7.2 7.4 7.9 6.4 6.3 6.1 7.7 6.3 6.4 6.0 6.9 6.7 6.9 5.8 6.8
[145] 6.7 6.7 6.3 6.5 6.2 5.9
```

You can also access data frame columns by number or name with bracket notation:

```
iris[,1]
## [1] 5.1 4.9 4.7 4.6 5.0 5.4 4.6 5.0 4.4 4.9 5.4 4.8 4.8 4.3 5.8 5.7 5.4 5.1
## [19] 5.7 5.1 5.4 5.1 4.6 5.1 4.8 5.0 5.0 5.2 5.2 4.7 4.8 5.4 5.2 5.5 4.9 5.0
## [37] 5.5 4.9 4.4 5.1 5.0 4.5 4.4 5.0 5.1 4.8 5.1 4.6 5.3 5.0 7.0 6.4 6.9 5.5
## [55] 6.5 5.7 6.3 4.9 6.6 5.2 5.0 5.9 6.0 6.1 5.6 6.7 5.6 5.8 6.2 5.6 5.9 6.1
## [73] 6.3 6.1 6.4 6.6 6.8 6.7 6.0 5.7 5.5 5.5 5.8 6.0 5.4 6.0 6.7 6.3 5.6 5.5
## [91] 5.5 6.1 5.8 5.0 5.6 5.7 5.7 6.2 5.1 5.7 6.3 5.8 7.1 6.3 6.5 7.6 4.9 7.3
## [109] 6.7 7.2 6.5 6.4 6.8 5.7 5.8 6.4 6.5 7.7 7.7 6.0 6.9 5.6 7.7 6.3 6.7 7.2
## [127] 6.2 6.1 6.4 7.2 7.4 7.9 6.4 6.3 6.1 7.7 6.3 6.4 6.0 6.9 6.7 6.9 5.8 6.8
## [145] 6.7 6.7 6.3 6.5 6.2 5.9
iris[, "Petal.Width"]
## [1] 0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 0.2 0.2 0.1 0.1 0.2 0.4 0.4 0.3
## [19] 0.3 0.3 0.2 0.4 0.2 0.5 0.2 0.2 0.4 0.2 0.2 0.2 0.2 0.4 0.1 0.2 0.2 0.2
## [37] 0.2 0.1 0.2 0.2 0.3 0.3 0.2 0.6 0.4 0.3 0.2 0.2 0.2 0.2 1.4 1.5 1.5 1.3
## [55] 1.5 1.3 1.6 1.0 1.3 1.4 1.0 1.5 1.0 1.4 1.3 1.4 1.5 1.0 1.5 1.1 1.8 1.3
## [73] 1.5 1.2 1.3 1.4 1.4 1.7 1.5 1.0 1.1 1.0 1.2 1.6 1.5 1.6 1.5 1.3 1.3 1.3
## [91] 1.2 1.4 1.2 1.0 1.3 1.2 1.3 1.3 1.1 1.3 2.5 1.9 2.1 1.8 2.2 2.1 1.7 1.8
## [109] 1.8 2.5 2.0 1.9 2.1 2.0 2.4 2.3 1.8 2.2 2.3 1.5 2.3 2.0 2.0 1.8 2.1 1.8
## [127] 1.8 1.8 2.1 1.6 1.9 2.0 2.2 1.5 1.4 2.3 2.4 1.8 1.8 2.1 2.4 2.3 1.9 2.3
## [145] 2.5 2.3 1.9 2.0 2.3 1.8
```

Rows of a data frame are accessed by number with bracket notation. As seen above, you can also select using logical tests.

```
iris[1,]
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1           5.1           3.5           1.4           0.2  setosa

# not run because output very long; try it on your machine!
iris[which(iris$Species == "setosa"),]
```

Unlike matrices, rows and columns of data frames are something different. This is because under the surface, a data frame is really a list, and what looks like a column of a data frame is really an element of the list. So, a row of a data frame is really a new data frame with one row. When you work with data frames in R, the data frame class and its associated methods offer a convenient way to interact with the underlying list structure.

```
class(iris[1,])
## [1] "data.frame"
class(iris[,1])
## [1] "numeric"
```

Practically, this means that if you select a column from a data frame you get a vector, and if you select a row (or rows) you get another data frame. So, if you want to operate on values within a row, you might need to use an `apply()` function.

```
mean(iris[1,1:4])          # apply function needed
## Warning in mean.default(iris[1, 1:4]): argument is not numeric or logical:
## returning NA
## [1] NA
apply(iris[1,1:4], 1, mean) # works!
##      1
## 2.55
sum(iris[1,1:4])           # works!
## [1] 10.2
```

Columns can be added to data frames by assigning them a value:

```
iris2 <- iris  # Make a spare copy
iris2$petal.area <- iris2$Petal.Length*iris2$Petal.Width
```

You can view the first few rows of a data frame with the command `head()`, and the last few rows with the command `tail()`. These are incredibly useful!

```
head(iris2)
##      Sepal.Length Sepal.Width Petal.Length Petal.Width Species petal.area
## 1           5.1         3.5         1.4         0.2   setosa      0.28
## 2           4.9         3.0         1.4         0.2   setosa      0.28
## 3           4.7         3.2         1.3         0.2   setosa      0.26
## 4           4.6         3.1         1.5         0.2   setosa      0.30
## 5           5.0         3.6         1.4         0.2   setosa      0.28
## 6           5.4         3.9         1.7         0.4   setosa      0.68
tail(iris2)
##      Sepal.Length Sepal.Width Petal.Length Petal.Width Species petal.area
## 145           6.7         3.3         5.7         2.5 virginica    14.25
## 146           6.7         3.0         5.2         2.3 virginica    11.96
## 147           6.3         2.5         5.0         1.9 virginica     9.50
## 148           6.5         3.0         5.2         2.0 virginica    10.40
## 149           6.2         3.4         5.4         2.3 virginica    12.42
## 150           5.9         3.0         5.1         1.8 virginica     9.18
```

Columns can be removed from a data frame by assigning them a value of `NULL`.

```
iris2$petal.area <- NULL
head(iris2)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

We'll explore more aspects of data frames in [Chapter 5](#).

3.2.3 Matrices and arrays

Sometimes you have a structured set of values that does not need to be stored as a data frame, or should not be stored as a data frame for ease of access. These situations call for matrices and arrays.

An **array** with two dimensions is a **matrix**. The dimensions of a matrix are its **rows** and **columns**. Matrices look a little like data frames, but internally they are very different. Unlike a data frame and like all arrays, a matrix can contain values of *only one type*. Matrices are necessary for any matrix operations (e.g., Leslie matrices) and for some function inputs. For most routine data analyses, data frames are easier to work with.

Matrices are created with the `matrix()` function. The matrix function must be provided with the values, the dimensions of the matrix, and the method for filling (by rows or by columns). If you don't know the values yet, you can just use NA or 0.

```
my.mat1 <- matrix(1:12,nrow=3,ncol=4)
my.mat1
##      [,1] [,2] [,3] [,4]
## [1,]    1    4    7   10
## [2,]    2    5    8   11
## [3,]    3    6    9   12

my.mat2 <- matrix(1:12,nrow=3,ncol=4,byrow=TRUE)
my.mat2
##      [,1] [,2] [,3] [,4]
## [1,]    1    2    3    4
## [2,]    5    6    7    8
## [3,]    9   10   11   12

my.mat3 <- matrix(NA,nrow=3,ncol=4)
my.mat3
##      [,1] [,2] [,3] [,4]
## [1,]   NA   NA   NA   NA
## [2,]   NA   NA   NA   NA
## [3,]   NA   NA   NA   NA
```

Just as with other objects, the values in a matrix can be extracted using bracket notation. Because matrices have two dimensions, you must specify indices for both dimensions. This is done using a comma , to separate the row indices and column indices within the brackets. Just as with data frames, row indices are before the comma; column indices are after the comma. Leaving the row index or the column index blank will select all rows or columns, respectively.

```
my.mat1[2,1]# row 2, column 1
## [1] 2
my.mat1[3,3]# row 3, column 3
## [1] 9
```

Entire rows or columns of a matrix can be extracted by leaving the other dimension blank. You must still include the comma within the brackets. Notice that both the rows and columns of a matrix are vectors, not a matrix. If you want to get a row or column as a 1 row or 1 column matrix, you must specifically request this.

```

# first row
my.mat1[1,]
## [1] 1 4 7 10
is.matrix(my.mat1[1,])
## [1] FALSE

# third column
my.mat1[,3]
## [1] 7 8 9

# convert extracted part to matrix
my.mat3 <- matrix(my.mat1[1,], nrow=1)
is.matrix(my.mat3)
## [1] TRUE

```

An array in R is usually only called an array if it has ≥ 3 dimensions. Data stored this way can make programming easier, if there is a clear relationship between the dimensions. Arrays are not often encountered by the average user. The example below has 3 “layers”, each of which has 4 rows and 2 columns.

```

my.array <- array(1:24,dim=c(4,2,3))
my.array

```

```
, , 1
```

	[,1]	[,2]
[1,]	1	5
[2,]	2	6
[3,]	3	7
[4,]	4	8

```
, , 2
```

	[,1]	[,2]
[1,]	9	13
[2,]	10	14
[3,]	11	15
[4,]	12	16

```
, , 3
```

	[,1]	[,2]
[1,]	17	21
[2,]	18	22
[3,]	19	23

```
[4,]    20    24
```

One very nifty (and potentially confusing) ability of arrays is the ability to be sliced into differently shaped pieces¹. Consider the examples below and see if you can work out why the results are what they are.

```
my.array[, , 1]
##      [,1] [,2]
## [1,]    1    5
## [2,]    2    6
## [3,]    3    7
## [4,]    4    8
my.array[, 1, ]
##      [,1] [,2] [,3]
## [1,]    1    9   17
## [2,]    2   10   18
## [3,]    3   11   19
## [4,]    4   12   20
my.array[1, , ]
##      [,1] [,2] [,3]
## [1,]    1    9   17
## [2,]    5   13   21
my.array[1, 2, ]
## [1]  5 13 21
my.array[, 2, 1:2]
##      [,1] [,2]
## [1,]    5   13
## [2,]    6   14
## [3,]    7   15
## [4,]    8   16
```

3.2.4 Lists

One of the most important kinds of R object is the **list**. A list can be thought of like a *series of containers or buckets*. Each bucket can contain completely unrelated things, or even other series of buckets. Because of this flexibility, lists are extremely versatile and useful in R programming. Many function outputs are really lists (e.g., the outputs of `lm()` or `t.test()`). Data frames are also lists.

¹My personal record is a 7-dimensional array, which I used in a community simulation model. The model included many species-, age-, and sex-specific demographic parameters that had to be extracted and summarized, and because of the size of the model an array-based solution was more computationally-efficient than a list- or data frame-based solution. I don't recommend building arrays that complex...it was a nightmare to work with.

Lists can be created using the function `vector()` and then filled later. The following example creates an empty list with 4 elements (i.e., buckets), then fills each element with something different.

```
my.list <- vector("list",length=4)
my.list[[1]] <- 1:10
my.list[[2]] <- c("a","b","c")
my.list[[3]] <- t.test(iris$Sepal.Length)
my.list[[4]] <- iris
my.list
## [[1]]
##  [1]  1  2  3  4  5  6  7  8  9 10
##
## [[2]]
## [1] "a" "b" "c"
##
## [[3]]
##
##  One Sample t-test
##
## data:  iris$Sepal.Length
## t = 86.425, df = 149, p-value < 2.2e-16
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  5.709732 5.976934
## sample estimates:
## mean of x
##  5.843333
##
##
## [[4]]
##      Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1           5.1         3.5         1.4         0.2    setosa
## 2           4.9         3.0         1.4         0.2    setosa
## 3           4.7         3.2         1.3         0.2    setosa
## 4           4.6         3.1         1.5         0.2    setosa
## 5           5.0         3.6         1.4         0.2    setosa
## 6           5.4         3.9         1.7         0.4    setosa
## 7           4.6         3.4         1.4         0.3    setosa
## 8           5.0         3.4         1.5         0.2    setosa
## 9           4.4         2.9         1.4         0.2    setosa
## 10          4.9         3.1         1.5         0.1    setosa
## 11          5.4         3.7         1.5         0.2    setosa
## 12          4.8         3.4         1.6         0.2    setosa
## 13          4.8         3.0         1.4         0.1    setosa
## 14          4.3         3.0         1.1         0.1    setosa
```

## 15	5.8	4.0	1.2	0.2	setosa
## 16	5.7	4.4	1.5	0.4	setosa
## 17	5.4	3.9	1.3	0.4	setosa
## 18	5.1	3.5	1.4	0.3	setosa
## 19	5.7	3.8	1.7	0.3	setosa
## 20	5.1	3.8	1.5	0.3	setosa
## 21	5.4	3.4	1.7	0.2	setosa
## 22	5.1	3.7	1.5	0.4	setosa
## 23	4.6	3.6	1.0	0.2	setosa
## 24	5.1	3.3	1.7	0.5	setosa
## 25	4.8	3.4	1.9	0.2	setosa
## 26	5.0	3.0	1.6	0.2	setosa
## 27	5.0	3.4	1.6	0.4	setosa
## 28	5.2	3.5	1.5	0.2	setosa
## 29	5.2	3.4	1.4	0.2	setosa
## 30	4.7	3.2	1.6	0.2	setosa
## 31	4.8	3.1	1.6	0.2	setosa
## 32	5.4	3.4	1.5	0.4	setosa
## 33	5.2	4.1	1.5	0.1	setosa
## 34	5.5	4.2	1.4	0.2	setosa
## 35	4.9	3.1	1.5	0.2	setosa
## 36	5.0	3.2	1.2	0.2	setosa
## 37	5.5	3.5	1.3	0.2	setosa
## 38	4.9	3.6	1.4	0.1	setosa
## 39	4.4	3.0	1.3	0.2	setosa
## 40	5.1	3.4	1.5	0.2	setosa
## 41	5.0	3.5	1.3	0.3	setosa
## 42	4.5	2.3	1.3	0.3	setosa
## 43	4.4	3.2	1.3	0.2	setosa
## 44	5.0	3.5	1.6	0.6	setosa
## 45	5.1	3.8	1.9	0.4	setosa
## 46	4.8	3.0	1.4	0.3	setosa
## 47	5.1	3.8	1.6	0.2	setosa
## 48	4.6	3.2	1.4	0.2	setosa
## 49	5.3	3.7	1.5	0.2	setosa
## 50	5.0	3.3	1.4	0.2	setosa
## 51	7.0	3.2	4.7	1.4	versicolor
## 52	6.4	3.2	4.5	1.5	versicolor
## 53	6.9	3.1	4.9	1.5	versicolor
## 54	5.5	2.3	4.0	1.3	versicolor
## 55	6.5	2.8	4.6	1.5	versicolor
## 56	5.7	2.8	4.5	1.3	versicolor
## 57	6.3	3.3	4.7	1.6	versicolor
## 58	4.9	2.4	3.3	1.0	versicolor
## 59	6.6	2.9	4.6	1.3	versicolor

## 60	5.2	2.7	3.9	1.4 versicolor
## 61	5.0	2.0	3.5	1.0 versicolor
## 62	5.9	3.0	4.2	1.5 versicolor
## 63	6.0	2.2	4.0	1.0 versicolor
## 64	6.1	2.9	4.7	1.4 versicolor
## 65	5.6	2.9	3.6	1.3 versicolor
## 66	6.7	3.1	4.4	1.4 versicolor
## 67	5.6	3.0	4.5	1.5 versicolor
## 68	5.8	2.7	4.1	1.0 versicolor
## 69	6.2	2.2	4.5	1.5 versicolor
## 70	5.6	2.5	3.9	1.1 versicolor
## 71	5.9	3.2	4.8	1.8 versicolor
## 72	6.1	2.8	4.0	1.3 versicolor
## 73	6.3	2.5	4.9	1.5 versicolor
## 74	6.1	2.8	4.7	1.2 versicolor
## 75	6.4	2.9	4.3	1.3 versicolor
## 76	6.6	3.0	4.4	1.4 versicolor
## 77	6.8	2.8	4.8	1.4 versicolor
## 78	6.7	3.0	5.0	1.7 versicolor
## 79	6.0	2.9	4.5	1.5 versicolor
## 80	5.7	2.6	3.5	1.0 versicolor
## 81	5.5	2.4	3.8	1.1 versicolor
## 82	5.5	2.4	3.7	1.0 versicolor
## 83	5.8	2.7	3.9	1.2 versicolor
## 84	6.0	2.7	5.1	1.6 versicolor
## 85	5.4	3.0	4.5	1.5 versicolor
## 86	6.0	3.4	4.5	1.6 versicolor
## 87	6.7	3.1	4.7	1.5 versicolor
## 88	6.3	2.3	4.4	1.3 versicolor
## 89	5.6	3.0	4.1	1.3 versicolor
## 90	5.5	2.5	4.0	1.3 versicolor
## 91	5.5	2.6	4.4	1.2 versicolor
## 92	6.1	3.0	4.6	1.4 versicolor
## 93	5.8	2.6	4.0	1.2 versicolor
## 94	5.0	2.3	3.3	1.0 versicolor
## 95	5.6	2.7	4.2	1.3 versicolor
## 96	5.7	3.0	4.2	1.2 versicolor
## 97	5.7	2.9	4.2	1.3 versicolor
## 98	6.2	2.9	4.3	1.3 versicolor
## 99	5.1	2.5	3.0	1.1 versicolor
## 100	5.7	2.8	4.1	1.3 versicolor
## 101	6.3	3.3	6.0	2.5 virginica
## 102	5.8	2.7	5.1	1.9 virginica
## 103	7.1	3.0	5.9	2.1 virginica
## 104	6.3	2.9	5.6	1.8 virginica

## 105	6.5	3.0	5.8	2.2	virginica
## 106	7.6	3.0	6.6	2.1	virginica
## 107	4.9	2.5	4.5	1.7	virginica
## 108	7.3	2.9	6.3	1.8	virginica
## 109	6.7	2.5	5.8	1.8	virginica
## 110	7.2	3.6	6.1	2.5	virginica
## 111	6.5	3.2	5.1	2.0	virginica
## 112	6.4	2.7	5.3	1.9	virginica
## 113	6.8	3.0	5.5	2.1	virginica
## 114	5.7	2.5	5.0	2.0	virginica
## 115	5.8	2.8	5.1	2.4	virginica
## 116	6.4	3.2	5.3	2.3	virginica
## 117	6.5	3.0	5.5	1.8	virginica
## 118	7.7	3.8	6.7	2.2	virginica
## 119	7.7	2.6	6.9	2.3	virginica
## 120	6.0	2.2	5.0	1.5	virginica
## 121	6.9	3.2	5.7	2.3	virginica
## 122	5.6	2.8	4.9	2.0	virginica
## 123	7.7	2.8	6.7	2.0	virginica
## 124	6.3	2.7	4.9	1.8	virginica
## 125	6.7	3.3	5.7	2.1	virginica
## 126	7.2	3.2	6.0	1.8	virginica
## 127	6.2	2.8	4.8	1.8	virginica
## 128	6.1	3.0	4.9	1.8	virginica
## 129	6.4	2.8	5.6	2.1	virginica
## 130	7.2	3.0	5.8	1.6	virginica
## 131	7.4	2.8	6.1	1.9	virginica
## 132	7.9	3.8	6.4	2.0	virginica
## 133	6.4	2.8	5.6	2.2	virginica
## 134	6.3	2.8	5.1	1.5	virginica
## 135	6.1	2.6	5.6	1.4	virginica
## 136	7.7	3.0	6.1	2.3	virginica
## 137	6.3	3.4	5.6	2.4	virginica
## 138	6.4	3.1	5.5	1.8	virginica
## 139	6.0	3.0	4.8	1.8	virginica
## 140	6.9	3.1	5.4	2.1	virginica
## 141	6.7	3.1	5.6	2.4	virginica
## 142	6.9	3.1	5.1	2.3	virginica
## 143	5.8	2.7	5.1	1.9	virginica
## 144	6.8	3.2	5.9	2.3	virginica
## 145	6.7	3.3	5.7	2.5	virginica
## 146	6.7	3.0	5.2	2.3	virginica
## 147	6.3	2.5	5.0	1.9	virginica
## 148	6.5	3.0	5.2	2.0	virginica
## 149	6.2	3.4	5.4	2.3	virginica

```
## 150          5.9          3.0          5.1          1.8 virginica
```

Another way to make a list is to make its elements first and combine them into a list, with function `list()`.

```
x <- runif(10)
y <- "zebra"
z <- matrix(1:12, nrow=3)
a <- list(x, y, z)
a
```

```
[[1]]
[1] 0.6875380 0.3153156 0.2944260 0.6342272 0.9418565 0.7363750 0.7353674
[8] 0.5405603 0.4439155 0.7152392
```

```
[[2]]
[1] "zebra"
```

```
[[3]]
      [,1] [,2] [,3] [,4]
[1,]     1     4     7    10
[2,]     2     5     8    11
[3,]     3     6     9    12
```

Optionally, you can name each element as you make the list.

```
b <- list(x=x, y=y, z=z)
b
```

```
$x
[1] 0.6875380 0.3153156 0.2944260 0.6342272 0.9418565 0.7363750 0.7353674
[8] 0.5405603 0.4439155 0.7152392
```

```
$y
[1] "zebra"
```

```
$z
      [,1] [,2] [,3] [,4]
[1,]     1     4     7    10
[2,]     2     5     8    11
[3,]     3     6     9    12
```

Much like vectors, lists can be extended by using concatenation:


```
a2 <- c(a,b)
a2
```

```
[[1]]
[1] 0.6875380 0.3153156 0.2944260 0.6342272 0.9418565 0.7363750 0.7353674
[8] 0.5405603 0.4439155 0.7152392
```

```
[[2]]
[1] "zebra"
```

```
[[3]]
      [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
```

```
$x
[1] 0.6875380 0.3153156 0.2944260 0.6342272 0.9418565 0.7363750 0.7353674
[8] 0.5405603 0.4439155 0.7152392
```

```
$y
[1] "zebra"
```

```
$z
      [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
```

Single elements of a list can be accessed with **double brackets** `[[ ]>`. As with other objects, elements can be selected by name or by index. If selecting by name, the `$` symbol can be used instead of double brackets.

```
a[[1]]
## [1] 0.6875380 0.3153156 0.2944260 0.6342272 0.9418565 0.7363750 0.7353674
## [8] 0.5405603 0.4439155 0.7152392
b[[2]]
## [1] "zebra"
b[["z"]]
##      [,1] [,2] [,3] [,4]
## [1,]    1    4    7   10
## [2,]    2    5    8   11
## [3,]    3    6    9   12
b$z
##      [,1] [,2] [,3] [,4]
```

```
## [1,] 1 4 7 10
## [2,] 2 5 8 11
## [3,] 3 6 9 12
```

Using **single brackets** to access a list will return a *list* of the requested elements, not the requested elements themselves. This can be a little confusing. Compare the following two commands and notice that the first returns a vector containing the numbers 1 through 10 (the first element of `my.list`), while the second returns a list with 1 element, and that element is a vector with the numbers 1 through 10.

```
my.list[[1]]
## [1] 1 2 3 4 5 6 7 8 9 10
my.list[1]
## [[1]]
## [1] 1 2 3 4 5 6 7 8 9 10
```

The reason for this confusion is that the single brackets notation allows you to extract multiple elements of the list at once to make a new list.

```
a2 <- a[1:2]
a2
```

```
[[1]]
[1] 0.6875380 0.3153156 0.2944260 0.6342272 0.9418565 0.7363750 0.7353674
[8] 0.5405603 0.4439155 0.7152392
```

```
[[2]]
[1] "zebra"
```

3.2.5 S3, S4, and other paradigms

We’ve seen so far that R is object-oriented. What is little appreciated is that R has at least three object-oriented paradigms: **S3**, **S4**, and **R5**. Most objects in R are S3 objects: data frames, matrices, vectors, etc. The R5 objects are more formally called “Reference Classes” and are still being developed. The S4 objects are only occasionally encountered by the average user, although some R developers believe that S4 objects should replace S3 in general use. Their reasons are esoteric and we won’t get into them here. In a nutshell, S4 classes are structured more formally than S3 classes and are better for validation and code testing.

One key difference in practice between S4 and S3 objects is that S4 objects contain **slots** or **attributes**, which are accessed using `@` instead of `$`. Within each slot, an S4 object can contain all sorts of data, often in list form.

Chapter 4

Data import and export

For a beginning R user, getting data into and out of R can seem like the hardest part. This is because R interacts with your computer in a way that you're probably not used to. Once you understand a few things like how your computer stores files and how to save in the correct format, things will get easier!

4.1 Working with your file system

4.1.1 Your file system

Your file system begins with the **root directory**. In Windows, these are the top folder of a “letter drive” like **C:**, **E:**, and so on. On Mac and Linux, the root directory is simply **/**. Users should not store files in the root directory without a very good reason.

Every folder in the file system is a **branch** or **subdirectory** of the root. This creates a **hierarchy**, where folders and files are nested within other folders (except the root, which is not contained by anything).

Each folder and file has a **name**. Names of files are also called **filenames** or **file names**.

4.1.2 Folder addresses

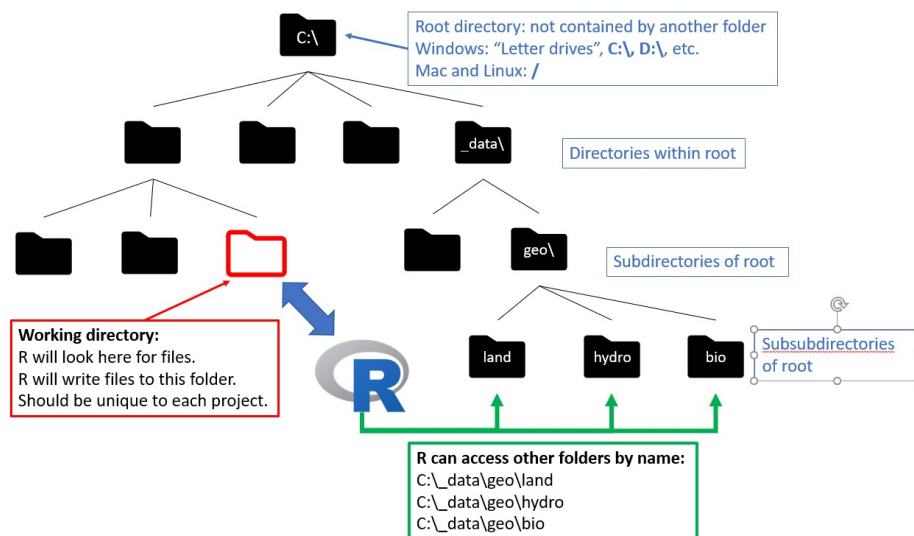
The directory path or just **path** is the “address” of a folder or file within the hierarchy. Paths are constructed by combining the names of the root, the containing folders, and finally the terminal name (for folders) or filename (for files). The elements of the path are separated by slashes:

- In Windows, elements of paths are separated by backslashes \.
- In Mac and Linux, elements of paths are separated by front-slashes /.

R uses front-slashes. So, Windows addresses need to have their \ changed to /, or escaped by doubling each to \\ (see figure below).

4.1.3 The working directory

Many programs, including R, will designate a folder as the **working directory**: a folder which contains the data the program can access, and which will contain any files created by the program. The default R working directory is the **Documents** folder on Windows, or the **Home** folder on a Mac. You should choose a different folder to be your working directory, however. One common practice is to create a single folder for each project which contains all the data needed for that project, and which will receive all the output from R for the project.



You can, and should, have a separate working directory for every project. This way files stay together, and the acts of importing and exporting files are easier.

In R, you can find your working directory in the console with `getwd()`:

```
getwd()
```

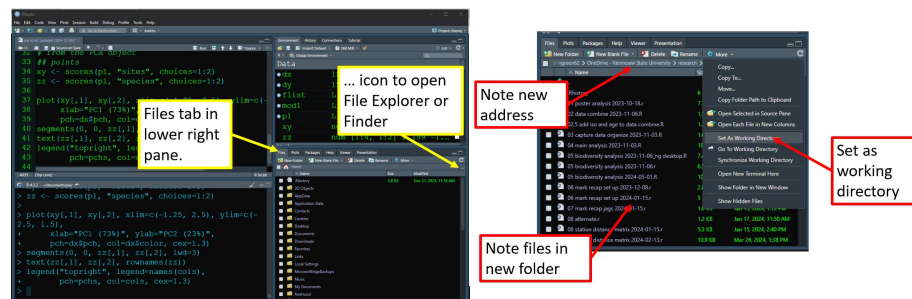
```
[1] "C:/Users/ngreen62/OneDrive - Kennesaw State University/biol 7410/_qq25"
```

You can change it with `setwd()`:

```
# example; not run
setwd("C:/Users/ngreen62/Documents/biol 7410")
```

These commands are usually run first in a script, to facilitate data import.

In RStudio, you can set your working directory in the Files tab by first navigating to a folder, then clicking **More - Set as working directory**.



What is the advantage of setting your working directory? R will look in the working directory **first** for any files, and put outputs there by default.

For example, the command `dir()` lists the contents of a folder. Without any arguments, it will automatically list the contents of the working directory.

```
# not run, but try on your machine:

# equivalent commands:
dir()
dir(getwd())

# for other folders, you need to specify the path:
dir("C:/Users/ngreen62/Documents/biol 7410/data")
```

When importing data, R will import from the working directory by file name, but requires a path for a file in any other folder.

```
# not run

# import from working directory:
my.data <- read.csv("example.csv")

# equivalent:
my.data <- read.csv(paste(getwd(), "example.csv", sep="/"))
# import from somewhere else:
my.data <- read.csv("C:/Users/ngreen62/Documents/example.csv")
```

If you open an R script with R or RStudio that is contained in a folder, that folder will be automatically set as the working directory. Clever use of the working directory can save you a lot of time. Sloppy use of the working directory (e.g., not specifying the working directory in each R or RStudio session) can lead to disaster and frustration.

4.2 Formatting data files

Before you try to import your data into R, take a few minutes to set yourself up for success.

4.2.1 Naming data files

Good file names are essential for a working biologist. Chances are you will work on many projects over the course of your career, each with its own collection of data files. Even if you store each project's data in a separate folder, it is a very good idea to give data files within those folders names that are descriptive. I have worked with many people who insist on using file names like “data.xlsx” and relying on their own memory or paper notes to tell them which file is which. Consider this list of particularly egregious file names:

- data.txt
- Data.txt
- data.set.csv
- analysis data.xlsx
- thesis data.xlsx
- DATA FINAL - copy.txt
- data current.revised.2019.txt.xlsx
- final thesis data revised revised final revised analysis.csv

We are all guilty of this kind of bad naming from time to time. What do all of the above have in common? They are vague. The names contain no meaningful information about what the files actually contain. Some of them are ambiguous: different operating systems or programs might read “data.txt” and “Data.txt” as the same file, or as different files!

Data file names should be short, simple, and descriptive. There are no hard and fast rules for naming data files so I'm not going to prescribe any. Instead, I'll share some examples and explain why these names are helpful. As you go along

and get better at working with data you will probably develop your own habits, and that's okay. Just try to follow the guiding principles of short, simple, and descriptive.

Here are some example file names with explanations of their desirable properties:

sturgeon_growth_data_2017-02-21.txt

This name declares what the file contains (growth data for sturgeon), and the date it was created. Notice that the date is given as “YYYY-MM-DD”. This format is useful because it automatically sorts chronologically. This file name separates its words with underscores rather than spaces or periods. This can prevent some occasional issues with programs misreading file names (white space and periods can have specific meanings in some contexts).

dat_combine_2020-07-13.csv

This name signifies that the file contains a “combined” dataset, and includes the date of its creation. This would be kept in a folder with the files that were combined to make it. Within a project I usually use prefixes on file names to help identify them: “dat” for datasets with response and explanatory variables; “met” for “metadata”; “geo” or “loc” for geospatial datasets and GIS layers; and “key” for lookup tables that relate dat and met files. You might develop your own conventions for types of data that occur in your work.

dat_vegcover_nrda_releaseversion_2021-05-03.csv

This name is similar to the last name, but includes some additional signifiers for the project (NRDA) and the fact that this is the final released version.

4.2.2 Setting up data files

Recently Broman and Woo (2018) reviewed good practices for storing data in spreadsheets. Their article is short and easy to read, and gave a few recommendations for data formatting:

- Be consistent
- Choose good names for things
- Write dates as YYYY-MM-DD
- No empty cells
- Put just one thing in a cell
- Make it a rectangle
- Create a data dictionary
- No calculations in the raw data files

- Do not use font color or highlighting as data
- Make backups
- Use data validation to avoid errors
- Save the data in plain text files

Most people who work with data, myself included, probably agree with their recommendations. Better, there is plenty of room for individual preferences in the exact implementation of each principle. You will save yourself a lot of headache if you follow their advice.

4.2.2.1 Variable names

Variable names, like data file names, should be brief but informative. There are few restrictions on what variable names can be in R. For simplicity, I recommend you use only lowercase letters, numbers, underscores (_), and periods (.) in variable names. Other symbols are either inconvenient to type, or can cause issues being parsed by R. Some common pitfalls include:

Leading numerals will cause an error if not enclosed in quotation marks:

```
x <- iris[1:5,]
names(x)[1] <- "3a"
x$3a
x$'3a'
x[, "3a"]
## Error in parse(text = input): <text>:3:3: unexpected numeric constant
## 2: names(x)[1] <- "3a"
## 3: x$3
##      ^
```

Numeral-only names will cause an error. They can also be misinterpreted as column numbers. Don't give your variables numeral-only names.

```
x <- iris[1:5,]
names(x)[1] <- "3"
x$3      # error
x$'3'    # works
x[,3]    # column number 3
x[, "3"] # column named "3"

## Error in parse(text = input): <text>:3:3: unexpected numeric constant
## 2: names(x)[1] <- "3"
## 3: x$3
##      ^
```


Spaces in variable names will cause errors because R interprets a space as whitespace, not part of a name. If you insist on using spaces, enclose the variable name in quotation marks every time you use it. This is one of the few places where R cares about whitespace.

```
x <- iris[1:5,]
names(x)[1] <- "a b"

# causes error:
x$a b

# works:
x$'a b'

## Error in parse(text = input): <text>:5:5: unexpected symbol
## 4: # causes error:
## 5: x$a b
##      ^
```

Special characters in variable names can cause issues with parsing. Many symbols that you might think to use, such as hyphens (aka: dashes, -) or slashes / are also R operators. Again, try to avoid these issues by using only letters, numbers, underscores, and periods.

```
x <- iris[1:5,]
names(x)[1] <- "a/b"
b <- 10
# error:
x$a/b
# works:
x$'a/b'

names(x)[1] <- "surv%"
x
# error:
x$surv%

# works:
x$'surv%'
## Error in parse(text = input): <text>:12:7: unexpected input
## 11: # error:
## 12: x$surv%
##      ^
```

Uppercase letters in variable names work just fine, but they can be inconvenient. Because R is case sensitive, you need to type the variable exactly the

same way each time. This can be tedious if you have several variables with similar names. In my code I prefer to use only lower case letters, so I don't ever have to remember how names are capitalized.

```
x <- iris[1:5,]
names(x)[1] <- "sep.len" # convenient
names(x)[2] <- "SEP.LEN" # slightly inconvenient
names(x)[3] <- "Sep.Len" # more inconvenient
names(x)[4] <- "SeP.lEn" # just don't
x
##   sep.len SEP.LEN Sep.Len SeP.lEn Species
## 1    5.1     3.5    1.4    0.2   setosa
## 2    4.9     3.0    1.4    0.2   setosa
## 3    4.7     3.2    1.3    0.2   setosa
## 4    4.6     3.1    1.5    0.2   setosa
## 5    5.0     3.6    1.4    0.2   setosa
```

4.2.2.2 Data values

All data in R have a type: numeric, logical, character, etc., as we learned in Section 3.1. Within a vector, matrix, or array, all values must have the same type or they will be converted to character strings. Each variable in a data frame (the most common R data structure) is essentially a vector. This means that in your input file you should try to have one and only one type of data in each column. Otherwise, R will coerce the values into character strings, which may or may not be a problem. Putting only one type of value into each column will also help get your data into “wide” or “tidy” format (Wickham 2014).

Entering data should also include some measure of **quality assurance and quality control (QA/QC)**. This basically means making sure that your data are entered correctly, and are formatted correctly. Correct entry can be accomplished by direct comparison to original datasheets or lab notebooks. Cross-checking is a valuable tool here: have two people enter the data and compare their files.

Data formatting requires thinking about what kinds of data are being entered. For numbers, this means entering the correct values and an appropriate number of significant digits. For text values, this may mean spell-checking or settling on a standard set of possible values. The figure below shows some problems that a QA/QC step might fix:

	A	B	C
1	species	tsn	common
2	Peromyscus leucopus	180278	white-footed mouse
3	Peromyscus leucopus	180278	White foot mouse
4	Peromyscus leucodus	180278	White_footed_mouse
5	Microtus ochrogaster	180312	Prairie_vole
6	Microtus ochrogaster	180312	Prairie vole
7	...	...	...

4.3 Data import

4.3.1 Importing from text files

Data for R should be stored in plain-text files. Two of the most popular options are tab-delimited files (**.txt**) and comma-delimited files (**.csv**). The “delimited” part means that some special character (like tab or comma) is used to separate values on a row of the file. Text files can be produced from Excel spreadsheets by going to **Save as**, selecting a folder, and then selecting “Text (Tab delimited)(*.txt)” or “CSV (Comma delimited)(*.csv)” from the dropdown menu for file type. Do *not* select “CSV UTF-8 (Comma delimited)(*.csv)” as this will cause issues in R.

4.3.2 Importing from saved workspaces

4.3.3 Importing special cases

4.4 Data export

4.4.1 Exporting text files

4.4.2 Saving workspaces

4.4.3 Exporting graphics

Chapter 5

Data manipulation with R

Brackets!

Part III

Exploratory data analysis with R

Blah blah test test test.

Chapter 6

Basic descriptive statistics

What do you mean, mean?

Chapter 7

Summarizing biological data

Blah blah test.

Chapter 8

Visualizing biological data

Blah blah test.

Chapter 9

Probability distributions for biologists

Just a little bit of calculus.

Chapter 10

Common problems in biological data

Mo data, mo problems.

Chapter 11

Planning your data analysis

What test do I do?

Part IV

Introductory statistics with R

Blah blah test test test.

Chapter 12

Introductory statistics with R

Everything you should have learned in Biostats.

Chapter 13

Tests comparing groups

All the t-tests.

Chapter 14

Tests for continuous patterns

Lines lines lines.

Chapter 15

Tests for contingency

It's not an X, it's a χ .

Part V

Generalized linear models (GLM)

Blah blah test test test.

Chapter 16

Introduction to generalized linear models (GLM)

A link function to the past.

Chapter 17

GLM for continuous response data

Still kind of linear.

Chapter 18

GLM for count data

1! Hah ha ha

2! Hah ha ha

Chapter 19

GLM for binary and proportional responses

Even our curves have curves.

It's what's under the curve that counts.

Part VI

Extensions of GLMs: nonlinear and mixed models

Blah blah test test test.

Chapter 20

Nonlinear models for biology

Things are not always linear.

Chapter 21

Random effects and mixed models

So random.

Part VII

Multivariate data analysis

Blah blah test test test.

Chapter 22

Introduction to multivariate data and data analysis

How far is too far?

Chapter 23

Clustering for biologists

Two is company, but three's a crowd.

Chapter 24

Ordination

We're all just points on a graph.

Chapter 25

Multivariate hypothesis testing

Anything is permuted.

Part VIII

Alternatives to p -values: information-theoretic and Bayesian methods

Blah blah test test test.

Chapter 26

Information theoretic model-based inference

Ain't AIC grand?

Chapter 27

Introduction to Bayesian inference for biologists

Bayes bayes bayes.

Part IX

Bonus material

Material that didn't quite fit anywhere else. Future chapters will likely be added here.

Chapter 28

Advanced R graphics

Base graphics are the best graphics.

Appendix A

Source and preparation of NEON data

Small mammal data.

- Bolker, Benjamin M. 2008. *Ecological Models and Data in R*. Princeton, New Jersey, USA: Princeton University Press.
- Broman, Karl W., and Kara H. Woo. 2018. “Data Organization in Spreadsheets.” *The American Statistician* 72 (1): 2–10. <https://doi.org/10.1080/00031305.2017.1375989>.
- Dalgaard, Peter. 2002. *Introductory Statistics with R*. Springer Science+Business Media.
- Green, Nicholas S, Mark L Wildhaber, and Janice L Albers. 2019. “Potential Responses of the Lower Missouri River Shovelnose Sturgeon (*Scaphirhynchus platyrhynchus*) Population to a Commercial Fishing Ban.” *Journal of Applied Ichthyology* 35 (1): 370–77. <https://doi.org/10.1111/jai.13701>.
- Green, Nicholas S, Mark L Wildhaber, Janice L Albers, Thomas W Pettit, and Michael J Hooper. 2020. “Efficient Mammal Biodiversity Surveys for Ecological Restoration Monitoring.” *Integrated Environmental Assessment and Management*. <https://doi.org/10.1002/ieam.4324>.
- James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. 2013. *An Introduction to Statistical Learning*. 7th ed. Vol. 112. Springer. <http://faculty.marshall.usc.edu/gareth-james/ISL/>.
- Kéry, Marc. 2010. *Introduction to WinBUGS for Ecologists: Bayesian Approach to Regression, ANOVA, Mixed Models and Related Analyses*. Academic Press.
- Kirchner, Brianna N, Nicholas S Green, David A Sergeant, Jeffrey N Mink, and Kenneth T Wilkins. 2011. “Responses of Small Mammals and Vegetation to a Prescribed Burn in a Tallgrass Blackland Prairie.” *The American Midland Naturalist* 166 (1): 112–25. <https://doi.org/10.1674/0003-0031-166.1.112>.

- Legendre, Pierre, and Louis Legendre. 2012. *Numerical Ecology*. Amsterdam: Elsevier.
- McCune, Bruce, James B Grace, and Dean L Urban. 2002. *Analysis of Ecological Communities*. MjM software design, Gleneden Beach, Oregon, USA.
- National Ecological Observatory Network (NEON). 2025. "Small Mammal Box Trapping (DP1.10072.001)." National Ecological Observatory Network (NEON). <https://doi.org/10.48443/YRH2-6058>.
- Ritz, Christian, Florent Baty, Jens C. Streibig, and Daniel Gerhard. 2015. "Dose-Response Analysis Using R." Edited by Yinglin Xia. *PLoS ONE* 10 (12): e0146021. <https://doi.org/10.1371/journal.pone.0146021>.
- Spector, Phil. 2008. *Data Manipulation with R*. New York: Springer Science+Business Media.
- Wasserstein, Ronald L, and Nicole A Lazar. 2016. "The ASA Statement on p-Values: Context, Process, and Purpose." *The American Statistician*. Taylor & Francis. <https://doi.org/10.1080/00031305.2016.1154108>.
- Wickham, Hadley. 2014. "Tidy Data." *Journal of Statistical Software* 59 (10). <https://doi.org/10.18637/jss.v059.i10>.
- Wickham, Hadley, Mara Averick, Jennifer Bryan, Winston Chang, Lucy D'Agostino McGowan, Romain François, Garrett Grolemond, et al. 2019. "Welcome to the tidyverse." *Journal of Open Source Software* 4 (43): 1686. <https://doi.org/10.21105/joss.01686>.
- Zuur, Alain F, Elena N Ieno, and Graham M Smith. 2007. *Analysing Ecological Data*. New York: Springer Science+Business Media.
- Zuur, Alain F, Elena N Ieno, Neil J Walker, Anatoly A Saveliev, Graham M Smith, et al. 2009. *Mixed Effects Models and Extensions in Ecology with R*. Vol. 574. New York: Springer.
- Zuur, Alain, Elena N Ieno, and Erik Meesters. 2009. *A Beginner's Guide to R*. New York: Springer Science+Business Media.